

**KLASIFIKASI JENIS PENYAKIT DAN HAMA DAUN
TANAMAN KAPAS MENGGUNAKAN METODE
*CONVOLUTIONAL NEURAL NETWORK***

SKRIPSI



OLEH:
ALEISYA ZAHARI SALAM
201011000103

**PROGRAM STUDI MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS PAMULANG
TANGERANG SELATAN
2024**

**KLASIFIKASI JENIS PENYAKIT DAN HAMA DAUN
TANAMAN KAPAS MENGGUNAKAN METODE
*CONVOLUTIONAL NEURAL NETWORK***

SKRIPSI

Diajukan sebagai salah satu syarat memperoleh gelar Sarjana Matematika



OLEH:

**ALEISYA ZAHARI SALAM
201011000103**

**PROGRAM STUDI MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS PAMULANG
TANGERANG SELATAN
2024**

SURAT PERNYATAAN

Saya yang bertanda tangan di bawah ini :

Nama : Aleisya Zahari Salam

NIM : 201011000103

Program Studi : Matematika

Fakultas : Matematika dan Ilmu Pengetahuan Alam

Menyatakan bahwa tugas akhir yang saya buat dengan judul:

**KLASIFIKASI JENIS PENYAKIT DAN HAMA DAUN TANAMAN
KAPAS MENGGUNAKAN METODE *CONVOLUTIONAL NEURAL
NETWORK***

1. Merupakan hasil karya ilmiah sendiri, bukan merupakan karya yang pernah diajukan untuk memperoleh gelar akademik oleh pihak lain, dan bukan merupakan hasil plagiat.
2. Saya ijin untuk dikelola oleh Universitas Pamulang sesuai dengan norma hukum dan etika yang berlaku.

Pamulang, Juli 2024

Yang menyatakan,

Materai 10.000

Aleisya Zahari Salam

NIM. 201011000103

LEMBAR PERSETUJUAN

KLASIFIKASI JENIS PENYAKIT DAN HAMA DAUN TANAMAN KAPAS MENGGUNAKAN METODE *CONVOLUTIONAL NEURAL NETWORK*

Oleh:

ALEISYA ZAHARI SALAM

201011000103

Telah diperiksa dan disetujui serta dianggap layak untuk diuji secara lisan melalui

Sidang Tugas Akhir oleh:

Pembimbing

Yulianti Rusdiana, S.Si., M.Sc.

NIDN. 0416078702

Ketua

Program Studi Matematika

Enggar Prasetyawan, S.Pd., M.Pd.

NIDN. 0414069002

LEMBAR PENGESAHAN

NIM : 201011000103
Nama : Aleisya Zahari Salam
Fakultas : Matematika dan Ilmu Pengetahuan Alam
Program Studi : Matematika S1
Judul Skripsi : Klasifikasi Jenis Penyakit Dan Hama Daun Tanaman
Kapas Menggunakan Metode *Convolutional Neural Network*

Menerangkan bahwa skripsi ini telah berhasil dipertahankan dalam sidang skripsi dan dinyatakan **LULUS**.

Tangerang Selatan, 19 Juli 2024

Penguji I

Penguji II

Usep Rahmat, M.Si
NIDN. 8882600016

Besse Arnawisuda Ningsi, S.Si, M.Si
NIDN. 0424018302

Pembimbing

Yulianti Rusdiana, S.Si., M.Sc.
NIDN. 0416078702

Mengetahui,
Ketua Program Studi Matematika

Enggar Prasetyawan, S.Pd., M.Pd.
NIDN. 0414069002

MOTTO DAN PERSEMBAHAN

Motto hidup saya adalah usaha tidak akan mengkhianati hasil. Selama saya memberikan upaya saya terhadap sesuatu sebaik mungkin, saya percaya hasil yang baik akan selalu mengikuti. Skripsi ini saya persembahkan kepada orang tua saya yang selalu mendukung dan percaya kepada saya, serta mendoakan keberhasilan saya.

DAFTAR RIWAYAT HIDUP

Data Pribadi:

- Nama : Aleisya Zahari Salam
- Tempat/Tanggal Lahir : Jakarta, 19 Oktober 2001
- Tempat Tinggal : Ciledug, Tangerang, Banten
- Jenis Kelamin : Perempuan
- Kewarganegaraan : Indonesia
- Agama : Islam
- Email : aleisyazahari19@gmail.com
- Telp. : 085889403125

Pendidikan Formal:

- MI Al-Husna (2008 - 2014)
- MTsN 13 Jakarta (2014 - 2017)
- SMAN 90 Jakarta (2017 - 2020)

Pengalaman Kerja

- *Project-Based virtual intern :*
IDE/X Parners x Rakamin Academy (2023 – 2024)
- *Project-Based virtual intern :*
Home Kredit x Rakamin Academy (2024)

LEMBAR PERNYATAAN PERSETUJUAN PUBLIKASI KARYA ILMIAH UNTUK KEPENTINGAN AKADEMIS

(Hasil Karya Perseorangan)

Sebagai Civitas Akademika Universitas Pamulang, saya yang bertanda tangan dibawah ini:

Nama : Aleisya Zahari Salam
NIM : 201011000103
Program Studi : Matematika
Fakultas : Matematika dan Ilmu Pengetahuan Alam
Jenis Karya : Skripsi

Demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Pamulang **Hak Bebas Royalti Non-Eksklusif (*Non-Exclusive Royalty Free Right*)** atas karya ilmiah saya yang berjudul: **KLASIFIKASI JENIS PENYAKIT DAN HAMA DAUN TANAMAN KAPAS MENGGUNAKAN METODE *CONVOLUTIONAL NEURAL NETWORK*** beserta perangkat yang ada (bila diperlukan). Dengan Hak Bebas Royalti Non-Eksklusif ini Universitas Pamulang berhak menyimpan, mengalih media/formatkan, mengelolanya dalam bentuk pangkalan data (*data base*), mendistribusikannya dan menampilkan/mempublikasikannya di internet atau media lain untuk kepentingan akademis tanpa perlu meminta ijin dari saya selama tetap tercantum nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta. Segala bentuk tuntutan hukum yang timbul atas pelanggaran Hak Cipta dalam karya ilmiah ini menjadi tanggung jawab saya pribadi.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Tangerang Selatan

Pada tanggal : 21 September 2024

Yang Menyatakan,

Aleisya Zahari Salam

KLASIFIKASI JENIS PENYAKIT DAN HAMA DAUN TANAMAN KAPAS MENGGUNAKAN METODE *CONVOLUTIONAL NEURAL NETWORK*

ALEISYA ZAHARI SALAM

201011000103

ABSTRAK

Kapas memiliki peran penting dalam industri tekstil dan pangan, sehingga budidayanya menjadi prioritas. Namun, kapas rentan terhadap berbagai penyakit dan hama. Identifikasi manual jenis penyakit dan hama daun tanaman kapas seringkali tidak akurat dan memakan waktu, sehingga diperlukan solusi yang lebih efisien. Pemanfaatan teknologi dan informasi mampu mengklasifikasi jenis penyakit dan hama daun tanaman kapas dengan pengolahan citra digital. Dengan adanya citra digital untuk menentukan jenis penyakit dan hama daun tanaman kapas dilakukan menggunakan metode *Convolutional Neural Network* (CNN). Penelitian ini bertujuan untuk mendapatkan hasil klasifikasi jenis penyakit dan hama pada daun tanaman kapas menggunakan metode CNN, dan mengetahui seberapa besar akurasi yang didapatkan pada klasifikasi tersebut. Data yang digunakan adalah gambar daun tanaman kapas dengan enam kategori, yaitu penyakit *Bacterial Blight*, *Powdery Mildew*, dan *Target spot*, hama *Aphids* dan *Army Worm*, serta kondisi sehat. Data dibagi menjadi tiga bagian, yaitu data latih, validasi, dan tes, dengan masing-masing rasio 75%;15%;10%. Data dilakukan *preprocessing* dan augmentasi. Proses model latih berakhir saat *epoch* 23 dengan *batch size* 16. Dari total 361 citra jenis penyakit dan hama tanaman, 330 citra terklasifikasi dengan benar. Proses klasifikasi menghasilkan akurasi sebesar 91%, ini menunjukkan kemampuan model yang baik dalam klasifikasi jenis penyakit dan hama daun tanaman kapas.

Kata Kunci: Convolutional neural network, klasifikasi jenis penyakit dan hama, daun tanaman kapas

CLASSIFICATION OF COTTON PLANT LEAF DISEASES AND PESTS USING *CONVOLUTIONAL NEURAL NETWORK* METHOD

ALEISYA ZAHARI SALAM

201011000103

ABSTRACT

Cotton plays an important role in the textile and food industries, making its cultivation a priority. However, cotton is susceptible to various diseases and pests. Manual identification of cotton leaf diseases and pests is often inaccurate and time-consuming, so a more efficient solution is needed. The utilization of technology and information is able to classify the types of diseases and pests of cotton leaves with digital image processing. With the existence of digital images to determine the types of diseases and pests of cotton plant leaves carried out using the Convolutional Neural Network (CNN) method. This research aims to obtain the results of the classification of disease and pest types on cotton plant leaves using the CNN method, and find out how much accuracy is obtained in the classification. The data used are images of cotton plant leaves with six categories, namely Bacterial Blight, Powdery Mildew, and Target spot diseases, Aphids and Army Worm pests, and healthy conditions. The data is divided into three parts, namely training, validation, and test data, with a ratio of 75%; 15%; 10% respectively. Data was preprocessed and augmented. The training model process ended at epoch 23 with a batch size of 16. Of the total 361 images of plant diseases and pests, 330 images were correctly classified. The classification process resulted in an accuracy of 91%, indicating the model's good ability to classify cotton leaf diseases and pests.

Keywords: *Convolutional neural network, classification of diseases and pests, cotton leaf.*

KATA PENGANTAR

Puji syukur Alhamdulillah kehadiran Allah SWT yang telah melimpahkan segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan penelitian tugas akhir yang merupakan salah satu persyaratan untuk menyelesaikan program studi strata satu (S1) pada Program Studi Matematika di Universitas Pamulang Tangerang Selatan.

Penulis menyadari tugas akhir ini masih jauh dari sempurna, karena itu kritik dan saran akan senantiasa penulis terima dengan senang hati. Dengan segala keterbatasan, penulis menyadari pula bahwa tugas akhir ini tidak akan terwujud tanpa bantuan, bimbingan, dan dorongan dari berbagai pihak. Untuk itu, dengan segala kerendahan hati, penulis menyampaikan ucapan terima kasih kepada:

1. Dr. E. Nurzaman AM, M.M. M.Si. selaku Rektor Universitas Pamulang.
2. Syaiful Bakhri, S.T., M.Eng.Sc., Ph.D. selaku Dekan Fakultas Matematika dan Ilmu Pengetahuan Alam (FMIPA) Universitas Pamulang.
3. Enggar Prasetyawan, S.Pd., M.Pd. selaku Ketua Program Studi Matematika.
4. Ibu Yulianti Rusdiana, S.Si., M.Sc. selaku pembimbing yang telah membimbing dalam penyusunan tugas akhir ini hingga selesai.
5. Bapak Usep Rahmat, M.Si. selaku Penguji I yang telah memberikan masukan dalam revisi setelah sidang.
6. Ibu Besse Arnawisuda Ningsi, S.Si., M.Si. selaku Penguji II yang telah memberikan masukan dalam revisi setelah sidang.
7. Bapak Abdus Salam dan Ibu Robiatul Adawiyah selaku orang tua penulis yang sudah membiayai penuh perkuliahan dan selalu memberikan segala bentuk dukungan kepada penulis.
8. Raffa dan Arya selaku adik penulis yang selalu menyemangati selama pengerjaan tugas akhir.

9. Semua pihak yang telah membantu dalam kelancaran tugas akhir ini yang tidak dapat saya sebutkan semua, terima kasih atas bantuan, dukungan, dan doa yang diberikan.

Akhir kata penulis hanya bisa berharap semoga tugas akhir ini dapat berguna dan bermanfaat bagi penulis dan pembaca sekalian walaupun masih jauh dari sempurna, untuk itu penulis sangat mengharapkan kritik dan saran yang membangun dikemudian hari dan semoga Allah SWT membalas kebaikan serta selalu mencurahkan hidayah serta taufik-Nya, Amin Ya Robbal ‘alamin.

Tangerang Selatan, Juli 2024

Penulis

DAFTAR ISI

HALAMAN SAMPUL	1
HALAMAN JUDUL	2
SURAT PERNYATAAN	iii
LEMBAR PERSETUJUAN	iv
LEMBAR PENGESAHAN	v
MOTTO DAN PERSEMBAHAN.....	vi
DAFTAR RIWAYAT HIDUP	vii
LEMBAR PERNYATAAN PERSETUJUAN PUBLIKASI	viii
ABSTRAK	viii
<i>ABSTRACT</i>	ix
KATA PENGANTAR.....	x
DAFTAR ISI.....	xii
DAFTAR GAMBAR.....	xiv
DAFTAR TABEL	xvi
DAFTAR LAMPIRAN	xvii
BAB I PENDAHULUAN.....	1
1.1. Latar Belakang	1
1.2. Identifikasi Masalah	3
1.3. Rumusan Masalah	3
1.4. Batasan Masalah.....	4
1.5. Tujuan Penelitian.....	4
1.6. Manfaat Penelitian.....	4
1.7. Sistematika Penulisan.....	4
BAB II LANDASAN TEORI	6
2.1. Penyakit dan Hama Tanaman Kapas	6
2.2. Machine Learning.....	9
2.3. <i>Convolutional Neural Network</i>	11
BAB III METODOLOGI PENELITIAN	23
3.1. Jenis Penelitian	23

3.2.	Variabel dan Parameter Penelitian	23
3.3.	Jenis dan Sumber Data	25
3.4.	Teknik Analisis Data	25
3.5.	Diagram Alir Penelitian.....	27
BAB IV HASIL DAN PEMBAHASAN		29
4.1.	Hasil Analisis Data	29
4.2.	Perancangan Model CNN.....	30
4.3.	Hasil Latih Model.....	40
4.4.	Hasil Uji Model	41
BAB V KESIMPULAN DAN SARAN		50
5.1.	Kesimpulan.....	50
5.2.	Saran	50
DAFTAR PUSTAKA		51
LAMPIRAN.....		54
LAMPIRAN 1		55
LAMPIRAN 2		57
LAMPIRAN 3		60
LAMPIRAN 4		62
LAMPIRAN 5		64

DAFTAR GAMBAR

Gambar 2.1 Daun Kapas.....	6
Gambar 2.2 Daun kapas yang terinfeksi <i>Bacterial Blight</i>	6
Gambar 2.3 Daun kapas yang terinfeksi <i>Powdery Mildew</i>	7
Gambar 2.4 Daun kapas yang terinfeksi <i>Target spot</i>	8
Gambar 2.5 Daun Kapas yang terserang hama <i>Aphids</i>	9
Gambar 2.6 Daun Kapas yang terserang hama <i>Army Worm</i>	9
Gambar 2.7 Operasi Convolution Sederhana.....	12
Gambar 2.8 Ilustrasi konvolusi setiap pergeseran.....	14
Gambar 2.9 Ilustrasi fungsi aktivasi ReLU	15
Gambar 2.10 Ilustrasi fungsi aktivasi softmax	16
Gambar 2.11 Ilustrasi <i>pooling layer</i>	17
Gambar 2.12 Struktur Dasar <i>Dense Layer</i>	17
Gambar 2.13 Unsur Dasar Parameter CNN.....	18
Gambar 2.14 Proses <i>Flattening</i>	20
Gambar 2.15 Ilustrasi Proses <i>Dropout</i>	20
Gambar 3.2 Diagram Alir Tahapan Penelitian.....	28
Gambar 4.1 Ringkasan Model Jaringan	30
Gambar 4.2 Tahap Ekstraksi fitur.....	31
Gambar 4.3 Convolution <i>Layer 1</i> (C1).....	31
Gambar 4.4 <i>Pooling Layer 1</i> (Pool 1)	33
Gambar 4.5 Convolution <i>Layer 2</i> (C2).....	34
Gambar 4.6 <i>Pooling Layer 2</i> (Pool 2)	35
Gambar 4.7 Tahap Klasifikasi.....	36
Gambar 4.8 Hasil akhir latih model	40
Gambar 4.9 Akurasi Proses Pelatihan Model.....	40
Gambar 4.10 <i>Loss</i> Proses Pelatihan Model	41
Gambar 4.11 Kode pengujian klasifikasi penyakit dan hama	43
Gambar 4.12 Tampilan Uji Coba Untuk Klasifikasi	43
Gambar 4.13 Hasil Klasifikasi Benar Terklasifikasi <i>Army Worm</i>	44
Gambar 4.14 Uji Coba Klasifikasi Menggunakan Data Tes	44
Gambar 4.15 Contoh beberapa sampel hasil klasifikasi.....	45

Gambar 4.16 Hasil Prediksi Jenis Penyakit dan Hama	46
Gambar 4.17 Laporan Klasifikasi <i>Precision</i> , <i>Recall</i> , dan <i>F1 Score</i>	48

DAFTAR TABEL

Tabel 2.1 <i>Confusion Matrix</i> Klasifikasi	21
Tabel 4.1 Daftar Teknik Data Augmentasi.....	29
Tabel 4.2 Hyperparameter <i>layer</i> C1	32
Tabel 4.3 Parameter <i>layer</i> C1	32
Tabel 4.4 Hyperparameter MaxPooling Layer 1 (Pool 1).....	34
Tabel 4.5 Hyperparameter <i>layer</i> C2	34
Tabel 4.6 Parameter <i>layer</i> C2	35
Tabel 4.7 Hyperparameter MaxPooling Layer 2 (Pool 2).....	36
Tabel 4.8 Jumlah Data Tes Setiap Kelas	41
Tabel 4.9 <i>Confusion Matrix</i> Data Tes	47
Tabel 4.10 Perhitungan manual <i>Precision</i> , <i>Recall</i> , dan <i>F1 score</i>	48

DAFTAR LAMPIRAN

Lampiran 1.1 <i>python import library</i>	55
Lampiran 1.2 <i>python persiapan data 1</i>	55
Lampiran 1.3 <i>python persiapan data 2</i>	56
Lampiran 2.1 <i>Python preprocessing dan augmentasi</i>	57
Lampiran 2.2 <i>Python ImageDataGenerator data train, validation, dan test</i>	58
Lampiran 2.3 <i>Python visualisasi data</i>	59
Lampiran 3.1 <i>Python rancangan model</i>	60
Lampiran 3.2 <i>Python Pelatihan Model</i>	60
Lampiran 3.3 <i>Python Visualisasi Proses Pelatihan Model</i>	61
Lampiran 4.1 <i>Python Uji Model Menggunakan Data Tes</i>	62
Lampiran 4.2 <i>Python Visualisasi Laporan Klasifikasi</i>	62
Lampiran 4.3 <i>Python Uji Manual</i>	62
Lampiran 5.1 <i>Website Sumber Data</i>	64
Lampiran 5.2 <i>Label Aphids</i>	64
Lampiran 5.3 <i>Label Army Worm</i>	65
Lampiran 5.4 <i>Label Healthy</i>	65
Lampiran 5.5 <i>Label Bacterial Blight</i>	66
Lampiran 5.6 <i>Label Powdery Mildew</i>	66
Lampiran 5.7 <i>Label Target Spot</i>	67

BAB I

PENDAHULUAN

1.1. Latar Belakang

Kapas merupakan salah satu tanaman terpenting di dunia. Menurut data *statista* oleh M. Shahbandeh (2024), produksi global mencapai sekitar 118,3 juta bal pada tahun 2022/2023. Tanaman ini merupakan sumber serat terbaru di dunia dan berperan penting dalam industri tekstil (Xiao dkk., 2019). Tidak hanya itu, tanaman ini juga menyediakan bahan baku berbagai produk seperti pakaian, minyak goreng, dan biofuel. Pentingnya kapas dalam perekonomian dunia sangat tinggi sehingga budidaya kapas menjadi prioritas utama.

Namun, tanaman kapas rentan terhadap berbagai penyakit dan serangan hama yang dapat mengganggu produksi selama pertumbuhannya (Chen dkk., 2020). Penyakit seperti *Bacterial Blight*, *Powdery Mildew*, dan *Target spot* dapat menyerang tanaman kapas, sementara hama seperti *Aphids* dan *Army Worm* juga menjadi ancaman yang serius. Dampak dari serangan penyakit dan hama ini sangat signifikan, sehingga menyebabkan kerugian bagi para petani. Kerugian ini tidak hanya mempengaruhi kuantitas hasil kapas tetapi juga dengan kualitas kapas yang dihasilkan.

Saat ini, mayoritas petani mengidentifikasi jenis penyakit dan hama tanaman kapas berdasarkan pengalaman atau saran dari ahli pertanian, metode ini sering kali memakan waktu dan tidak selalu akurat (Barbedo, 2016). Para petani memiliki kecenderungan untuk mengandalkan pengamatan secara visual, yang menyebabkan terbatasnya mendeteksi penyakit dan hama pada tahap awal perkembangannya. Kesalahan dalam identifikasi penyakit dan hama juga bisa menyebabkan petani menggunakan penanganan yang tidak tepat, seperti pestisida, sehingga berdampak negatif pada hasil kapas (Dong dkk., 2021). Oleh karena itu, diperlukannya pendekatan yang dapat meningkatkan efisiensi dan akurasi identifikasi untuk mengatasi tantangan ini, sehingga para petani mampu melakukan pencegahan sedini mungkin.

Perkembangan teknologi saat ini di sektor pertanian telah mengalami kemajuan. Implementasi teknologi pada sektor ini memberikan jalan lebih cerah, penuh dengan peluang dan keberlanjutan. Salah satu solusi yang menjawab tantangan identifikasi penyakit dan hama pada tanaman kapas adalah penggunaan sistem berbasis *machine learning*. Saat ini, banyak aplikasi *machine learning* dalam sektor pertanian yang diperlukan, di mana data dengan jumlah yang besar dapat dianalisis (Mishra dkk., 2016).

Penerapan teknologi *machine learning*, yaitu *Convolutional Neural Network* (CNN), memberikan solusi inovatif yang mampu meningkatkan akurasi identifikasi penyakit dan hama pada tanaman kapas. Keunggulan utama CNN adalah kapabilitasnya yang secara otomatis mengidentifikasi banyak fitur penting tanpa perlu adanya pengawasan manusia, sehingga menjadi metode yang paling umum digunakan (Alzubaidi dkk., 2021). CNN mampu mengekstrak dari citra daun tanaman kapas dengan cepat dan akurat untuk melakukan identifikasi. Sehingga, petani mampu meminimalisir kerugian yang dapat diakibatkan oleh penyebaran penyakit dan hama dibandingkan dengan melakukan klasifikasi secara manual.

Beberapa penelitian yang sudah dilakukan sebelumnya telah menunjukkan efektivitas penggunaan CNN dalam klasifikasi penyakit dan hama tanaman. Seperti penelitian yang dilakukan oleh Thakkar dkk. (2023) berjudul *Disease and Pest Detection in crops using Computer Vision: A Comprehensive Study*. Penelitian ini dilakukan dengan tujuan membangun model *machine learning* yang efektif agar dapat mendeteksi penyakit dan hama tanaman kapas. Penelitian ini menggunakan metode CNN dengan membandingkan model VGG16, ResNet50, dan Inception V3, dengan masing-masing mendapatkan akurasi 99.58%, 85.03%, dan 95.38%.

Penelitian yang dilakukan Qadir dkk. (2022) berjudul *A Novel-Based Deep Learning Approach for Early Diagnosis of Cotton Crop Disease*. Penelitian ini dilakukan untuk melakukan diagnosis awal pada penyakit tanaman kapas menggunakan metode *Convolutional Neural Network*. Model yang dilatih mendapatkan 99.08% akurasi dan 100% pada validasi.

Penelitian dengan judul *Performance Analysis of Deep Transfer Learning Models for the Automated Detection of Cotton Plant Diseases* yang dilakukan oleh Anwar dkk. (2023) melakukan penelitian dengan tujuan mendeteksi awal dari penyakit tanaman kapas yaitu *bacterial blight* dan *curl virus* secara otomatis. Model yang digunakan yaitu DenseNet121, DenseNet169, MobileNetV2, ResNet50V2, VGG16, dan VGG19, dengan model yang memiliki performa tertinggi yaitu DenseNet169 dan ResNet50V2 dengan akurasi sebesar 96%.

Berdasarkan latar belakang dan penelitian terdahulu di atas, dilakukan penelitian yang berjudul “Klasifikasi Jenis Penyakit dan Hama Daun Tanaman Kapas Menggunakan Metode *Convolutional Neural Network*”.

1.2. Identifikasi Masalah

Berdasarkan uraian latar belakang di atas, masalah yang teridentifikasi adalah sebagai berikut:

1. Berbagai penyakit seperti *Bacterial Blight*, *Powdery Mildew*, dan *Target spot* dapat menyerang tanaman kapas di berbagai fase pertumbuhan tanaman kapas. Hama seperti *Aphids* dan *Army Worm* juga dapat merusak daun dan menghambat produksi kapas. Hal ini dapat mengakibatkan kerugian ekonomi bagi para petani kapas.
2. Identifikasi penyakit dan hama secara manual oleh petani sering kali tidak efisien dan tidak akurat. Pengamatan visual yang dilakukan oleh petani memerlukan waktu yang lama dan sering kali terlambat dalam mendeteksi masalah pada tahap awal, yang menyebabkan penanganan yang tidak efektif dan kerugian lebih besar.
3. Kesalahan dalam identifikasi penyakit dan hama dapat menyebabkan petani menggunakan penanganan yang tidak tepat, seperti pestisida, sehingga berdampak negatif pada hasil kapas.

1.3. Rumusan Masalah

Berdasarkan latar belakang dan identifikasi masalah di atas, maka didapat rumusan masalah sebagai berikut ini:

1. Bagaimana hasil klasifikasi jenis penyakit dan hama daun tanaman kapas menggunakan metode *Convolutional Neural Network*?
2. Seberapa besar hasil akurasi klasifikasi jenis penyakit dan hama daun tanaman kapas menggunakan metode *Convolutional Neural Network*?

1.4. Batasan Masalah

Untuk menghindari permasalahan semakin melebar maka perlu batasan masalah, yaitu data jenis penyakit dan hama daun tanaman kapas tahun 2023.

1.5. Tujuan Penelitian

Berdasarkan rumusan masalah di atas maka tujuan dari penelitian ini adalah untuk:

1. Mendapatkan hasil klasifikasi jenis penyakit dan hama daun tanaman kapas menggunakan metode *Convolutional Neural Network*.
2. Mengetahui seberapa besar hasil akurasi klasifikasi jenis penyakit dan hama daun tanaman kapas menggunakan metode *Convolutional Neural Network*.

1.6. Manfaat Penelitian

Adapun manfaat yang dapat diberikan dalam penelitian ini adalah sebagai berikut:

1. Manfaat penelitian ini secara teoritis diharapkan mampu menambah wawasan dan pengetahuan dalam metode *Convolutional Neural Network* (CNN) dalam klasifikasi tanaman dan hama tanaman kapas.
2. Dari penelitian ini diharapkan dapat menyumbang pemikiran terhadap permasalahan yang berkaitan dengan klasifikasi tanaman dan hama tanaman kapas menggunakan metode CNN.
3. Hasil penelitian ini diharapkan dapat dijadikan sebagai landasan untuk pengembangan aplikasi berbasis kecerdasan buatan yang dapat membantu petani dalam mendiagnosis penyakit dan hama tanaman kapas secara cepat dan akurat.

1.7. Sistematika Penulisan

Adapun sistematika penulisan skripsi ini terdiri dari 5 (lima) bab diantaranya:

BAB I PENDAHULUAN

Pada bab ini berisikan latar belakang, identifikasi masalah, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian dan sistematika penulisan.

BAB II LANDASAN TEORI

Pada bab ini menguraikan pustaka mengenai landasan teori, dipaparkan teori-teori serta pustaka yang dipakai pada waktu penelitian.

BAB III METODOLOGI PENELITIAN

Pada bab ini berisikan tentang jenis penelitian, jenis dan sumber data, teknik analisis data, dan diagram alir penelitian.

BAB IV HASIL DAN PEMBAHASAN

Pada bab ini berisikan hasil serta pembahasan analisis dari penelitian yang telah dilakukan.

BAB V PENUTUP

Bab ini berisikan kesimpulan berdasarkan penelitian yang telah dilakukan dan saran mengenai pengembangan penelitian untuk selanjutnya.

BAB II

LANDASAN TEORI

2.1. Penyakit dan Hama Tanaman Kapas

Tanaman kapas, dikenal dengan nama ilmiah *Gossypium hirsutum* L. adalah tanaman penghasil serat kapas yang digunakan dalam pembuatan tekstil. Tanaman ini merupakan salah satu komoditas yang penting untuk industri tekstil (Xiao dkk., 2019). Memiliki akar tunggang yang panjang dan dalam hingga bisa melebihi panjang yang dimiliki batangnya. Waktu panen dibutuhkan dalam keadaan yang kering dan tidak dianjurkan untuk ditanam dalam wilayah yang memiliki curah hujan cukup tinggi.



Gambar 2.1 Daun Kapas

2.1.1 *Bacterial Blight*

Bacterial Blight atau bisa juga disebut sebagai penyakit hawar disebabkan oleh bakteri *Xanthomonas campestris* pv. *Malvacearum*. Penyakit ini merupakan penyakit yang dapat menyebabkan kerusakan besar pada daerah penanaman kapas (Toscano-Miranda dkk., 2022).



Gambar 2.2 Daun kapas yang terinfeksi *Bacterial Blight*

Gejala dari penyakit ini dapat terlihat pada Gambar 2.2. Bercak berwarna kuning pada awal kemunculan, kemudian akan menjadi cokelat kehitaman pada batang daun yang akan meluas dan menyebabkan kematian jaringan yang luas. Penyebaran penyakit ini sangat mudah saat terjadinya hujan berangin atau melalui irigasi (Cox dkk., 2019). Percikan air, terutama saat hujan memungkinkan bakteri untuk berpindah dari satu tempat ke tempat lainnya (Hakim, 2022). Penanganan yang tepat dan cepat dapat mengurangi dampak yang mungkin terjadi akibat penyakit ini.

2.1.2 *Powdery Mildew*

Penyakit *Powdery Mildew* yang diterjemahkan ke dalam Bahasa Indonesia menjadi penyakit tepung, yaitu penyakit pada tanaman yang disebabkan patogen tumbuh terlihat pada permukaan tanaman. *Powdery Mildew* mempunyai miselium berwarna putih dan spora yang terikat pada konidiofor, sehingga proses perkecambahannya tidak membutuhkan air bebas (Sastrahidayat, 2017). Hal ini memungkinkan penyebaran penyakit pada tanaman kapas mudah terjadi dan harus untuk segera ditangani. Gejala yang dialami dapat dilihat pada Gambar 2.3, pada permukaan luar tanaman yaitu daun ditemukan lapisan putih yang seperti tepung.



Gambar 2.3 Daun kapas yang terinfeksi *Powdery Mildew*

2.1.3 *Target Spot*

Penyakit *Target spot* (penyakit bercak) disebabkan oleh jamur *Corynespora cassiicola*. Penyakit ini semakin menjadi perhatian utama dalam produksi beberapa tanaman yang memiliki nilai ekonomi tinggi (N. Rondon & Lawrence, 2021). Gejala yang timbul

pada daun yaitu bercak berwarna coklat gelap di daun dengan ukuran yang beragam seperti halnya pada Gambar 2.4. Bercak ini berkembang secara progresif, menyebar pada daun yang terinfeksi sehingga daerah daun yang terjangkit tersebut akan mengering, layu dan akhirnya mati. Dampak dari penyakit ini cukup serius, sehingga penanganan harus dengan cepat dilakukan pada tanaman yang terjangkit penyakit ini.



Gambar 2.4 Daun kapas yang terinfeksi *Target spot*

2.1.4 *Aphids*

Aphids atau kutu daun merupakan salah satu serangga hama yang sering menyebabkan para petani secara signifikan kehilangan hasil pertanian mereka, seperti yang ditunjukkan pada Gambar 2.5. Memiliki ukuran Panjang 1-3 mm, hama ini akan mengisap nutrisi tanaman terutama pada bagian pucuk tunas dan bunga. Dengan cara mengisap nutrisi dari jaringan tanaman, hama ini melemahkan pertumbuhan tanaman tersebut, dan bagian daun yang terinfeksi akan keriput lalu berubah warnanya menjadi kuning dan terpuntir. *Aphids* tidak hanya mengisap nutrisi yang ada dalam tanaman, namun juga menjadi penyebab dari tersebarnya virus bagi tanaman (Nurdin, 2017). Penularan virus ini terjadi karena hama ini mengisap tanaman yang telah terinfeksi, yang kemudian menularkannya kepada tanaman yang masih sehat. Identifikasi dan penanganan hama ini harus dapat dilakukan dengan cepat agar mampu meminimalisir kerugian yang diakibatkannya.



Gambar 2.5 Daun Kapas yang terserang hama *Aphids*

2.1.5 Army Worm

Army Worm (Spodotera sp.) atau disebut juga dengan ulat gerayak, merupakan hama perusak pada tanaman terutama pada bagian daun. Populasi hama ini pada umumnya meningkat selama musim hujan dan ditandai dengan munculnya lubang pada daun tanaman yang menjadi gejala serangan hama ini. Saat dalam fase larva sudah dapat menyerang lapisan luar daun, dan jika sudah dewasa dapat menyerang seluruh bagian daun, ranting, batang muda, bunga, dan tanaman (Husen dkk., 2021). Hama ini biasanya menyerang pada tanaman yang berada pada fase pertumbuhan awal dan dapat mengakibatkan kerugian yang besar bagi petani. Sehingga, hama ini harus segera dilakukan penanganan, karena akan merusak tanaman dan hanya menyisakan tulang seperti yang terdapat pada Gambar 2.6.



Gambar 2.6 Daun Kapas yang terserang hama *Army Worm*

2.2. Machine Learning

Machine Learning (ML) merupakan bagian dari kecerdasan buatan yang mampu menyelesaikan permasalahan yang rumit untuk ditangani menggunakan bahasa pemrograman tradisional (Campesato, 2020). Tujuannya yaitu mengembangkan algoritma model yang memiliki

kemampuan mengidentifikasi pola atau memprediksi berdasarkan data yang diberikan tanpa harus menulis kode secara spesifik. Pada konsep dasarnya, ML memberikan kemampuan kepada sistem komputer untuk meningkatkan kinerjanya seiring berjalannya waktu tanpa perlu dilakukannya perubahan maupun perbaharui secara manual oleh *programmer*. Hal ini dilakukan dengan memanfaatkan data serta pengalaman sebagai sumber pembelajaran ML. *Machine Learning* terbagi menjadi tiga jenis utama, yaitu *Supervised Learning*, *Unsupervised Learning*, dan *Reinforcement Learning*.

2.2.1. *Supervised Learning*

Supervised Learning dalam proses pembelajarannya, mesin mempelajari berdasarkan data latih yang diterima dengan labelnya (Id, 2021). *Machine Learning* (ML) jenis ini berkonsentrasi pada proses pembelajaran terhadap data yang memiliki label atau keluaran yang diinginkan. Salah satu penggunaannya yaitu pada sistem klasifikasi emosi pada foto manusia yang terdiri atas kelas senang, marah, dan netral, di mana model akan mengelompokkan data baru berdasarkan data yang sebelumnya dipelajari. CNN adalah salah satu algoritma yang menerapkan *supervised learning*.

2.2.2. *Unsupervised Learning*

Unsupervised Learning merupakan kebalikan dari *supervised learning* dimana proses pembelajaran yang dilakukan mesin tanpa adanya petunjuk (Id, 2021). ML jenis ini tidak memerlukan label karena model akan belajar dari pola data yang diberikan untuk membuat kesimpulan. Tujuan utama jenis ML ini yaitu untuk mengekstrak pola atau struktur dalam data tanpa label atau petunjuk yang mengarahkan model. Salah satu contoh dari algoritma *Unsupervised Learning* yaitu pengelompokan (*clustering*).

2.2.3. *Reinforcement Learning*

Reinforcement Learning adalah metode pembelajaran mesin di mana agen mempelajari cara melakukan tindakan di lingkungannya untuk mencapai *tujuan* tertentu. *Machine Learning* jenis ini memungkinkan agen menerima umpan balik positif atau negatif

berdasarkan tindakan yang dilakukannya, memungkinkan agen belajar memilih tindakan terbaik dalam situasi tertentu. Metode ini digunakan dalam berbagai aplikasi seperti permainan komputer, pengendalian robot, dan sistem manajemen gudang.

2.3. *Convolutional Neural Network*

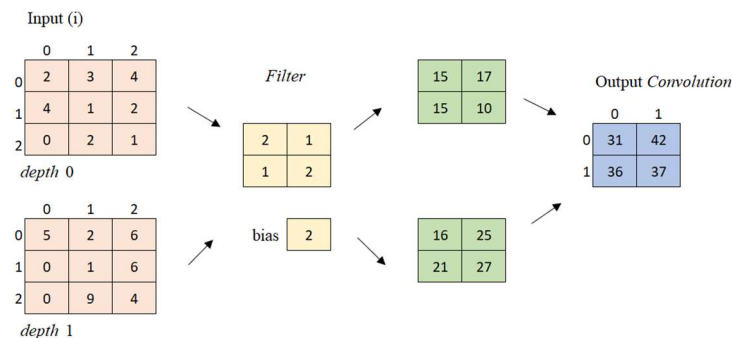
Convolutional Neural Network (CNN) merupakan tipe algoritma bagian *Deep Learning* yang mampu menerima masukan (*input*) dalam bentuk gambar, mengidentifikasi objek yang terdapat dalam sebuah gambar sehingga dapat digunakan mesin untuk belajar mengenali dan membedakan satu gambar dengan yang lainnya. Menurut Putro dkk. (2020) struktur CNN terdiri dari *input*, proses ekstraksi fitur, proses klasifikasi, dan *output*. *Input* yaitu data yang dimasukkan ke model, proses ekstraksi fitur berupa *layer konvolusi*, proses klasifikasi terdiri atas jaringan menyerupai neuron yang saling terhubung, dan *output* berupa hasil klasifikasi. CNN terdiri atas neuron yang memiliki *weight*, bias, dan fungsi aktivasi, dan termasuk ke dalam kategori *Deep Neural Network* disebabkan dalamnya tingkat jaringan serta banyak yang dilakukan implementasi dalam data citra. Tidak hanya itu, namun CNN juga terdiri atas neuron yang telah disusun dengan sedemikian rupa sehingga membentuk sebuah filter yang panjang, dan akan mendapatkan informasi representatif baru dari hasil perkalian dengan filter yang digunakan.

Pada umumnya, CNN menggunakan metode konvolusi dengan memindahkan sebuah *kernel* konvolusi (filter) dengan ukuran yang telah ditentukan pada seluruh gambar. Dengan melakukan hal tersebut, komputer mampu memperoleh informasi representatif yang baru melalui hasil perkalian gambar tersebut dengan filter yang diterapkan. Konvolusi CNN melibatkan pemindahan filter konvolusi tertentu di sepanjang gambar. Saat filter bergerak, operasi perkalian dilakukan pada nilai piksel dalam area inspeksi untuk menghasilkan hasil konvolusi. Setelah proses ini dilakukan pada seluruh gambar, komputer mengumpulkan informasi representatif baru yang mencerminkan pola dan fitur penting yang ada dalam gambar. Filter ini memungkinkan CNN mengekstrak dan memahami aspek tertentu dalam

gambar, seperti tepian, tekstur, dan fitur visual lainnya. Oleh karena itu, teknik konvolusi ini menjadi dasar kemampuan CNN dalam mengolah dan memahami data gambar dengan lebih efektif.

2.3.1. Convolution Layer

Convolution layer merupakan tahap pertama yang dilalui saat melakukan proses pembelajaran. *Layer* ini terdiri dari neuron-neuron yang tersusun secara terstruktur sehingga membentuk sebuah filter dengan ukuran tertentu (dalam piksel) (Batubara & Awangga, 2020). Parameter yang dimiliki dari *Convolution layer* yaitu *filters*, *striders*, dan *padding*. Parameter *filters* dalam proses *convolution* menentukan banyaknya *feature maps* yang akan dihasilkan. *Hyperparameter* ukuran *kernel* menentukan area data *input* yang akan diproses oleh *filter*. *Stride* menentukan berapa jarak *kernel* bergeser. Kemudian *padding* akan menambahkan sebuah nilai nol pada bagian tepi terluar dari *input*.



Gambar 2.7 Operasi Convolution Sederhana

Gambar 2.7 merupakan gambaran secara sederhana dari *convolution* pada CNN. Nilai $oc_{i,j}$ (*output convolution*) dalam konteks ini mengacu pada *output* atau hasil akhir dari hasil operasi konvolusi, di mana i mewakili kolom dan j mewakili baris. Kotak merah mewakili dari data *input* citra yang akan diproses, *input* ini memiliki 2 *depth* atau kedalaman data. Kotak kuning adalah *kernel* yang akan bergerak dari sudut kiri atas kemudian menuju sudut kanan bagian bawah dari *input* data, setiap *depth* akan melalui filter dan mendapatkan *output* tersendiri dalam kotak hijau. Hasil dari filter tersebut berada dalam kotak biru, di mana hasil filter dari masing-

masing *depth* akan dijumlahkan. Berikut adalah urutan matematis dari perhitungan operasi *convolution*.

$$oc_{i,j} = \sum_{c=0}^{N_c-1} \sum_{m=0}^{N_m-1} \sum_{n=0}^{N_n-1} I_{(i+m,j+r,c)} \times C_{1(m,n,c)} + bias \quad (2.1)$$

$$\begin{aligned} oc_{0,0} &= \sum_{c=0}^1 \sum_{m=0}^1 \sum_{n=0}^1 I_{(i+m,j+r,c)} \times C_{1(m,n,c)} + bias \\ &= \sum_{c=0}^1 \sum_{m=0}^1 I_{(0+0,0+0,0)} \times C_{1(0,0,0)} + I_{(0+0,0+1,0)} \times C_{1(0,1,0)} \\ &\quad + 2 \\ &= \sum_{c=0}^1 \sum_{m=0}^1 I_{(0,0,0)} \times C_{1(0,0,0)} + I_{(0,0+1,0)} \times C_{1(0,1,0)} + 2 \\ &= \sum_{c=0}^1 I_{(0,0,0)} \times C_{1(0,0,0)} + I_{(0,0+1,0)} \times C_{1(0,1,0)} \\ &\quad + I_{(0+1,0,0)} \times C_{1(1,0,0)} \\ &\quad + I_{(0+1,0+1,0)} \times C_{1(1,1,0)} + 2 \\ &= \left(\left(I_{(0,0,0)} \times C_{1(0,0,0)} + I_{(0,1,0)} \times C_{1(0,1,0)} \right. \right. \\ &\quad \left. \left. + I_{(1,0,0)} \times C_{1(1,0,0)} + I_{(1,1,0)} \times C_{1(1,1,0)} \right) \right. \\ &\quad \left. + 2 \right) + (I_{(0,0,1)} \times C_{1(0,0,0)} \\ &\quad + I_{(0,1,1)} \times C_{1(0,1,0)} + I_{(0+1,0,1)} \times C_{1(1,0,0)} \\ &\quad + I_{(1,1,1)} \times C_{1(1,1,0)}) + 2 \\ &= ((2 \times 2 + 3 \times 1 + 4 \times 1 + 1 \times 2) + 2) \\ &\quad + ((5 \times 2 + 2 \times 1 + 0 \times 1 + 1 \times 2) + 2) \\ &= ((4 + 3 + 4 + 2) + 2) + ((10 + 2 + 0 + 2) + 2) \\ &= 15 + 16 \\ &= 31 \end{aligned}$$

2	3	4			2	3	4		15	17
4	1	2			4	1	2			
0	2	1			0	2	1			

2	3	4			2	3	4		15	17
4	1	2			4	1	2		15	
0	2	1			0	2	1			10

Gambar 2.8 Ilustrasi konvolusi setiap pergeseran

Parameter dan ukuran akhir yang didapat dalam layer konvolusi dapat dihitung dengan persamaan berikut:

$$\text{Total Parameter} = F_c \times I_c (F_k \times F_l) + \text{bias} \quad (2.2)$$

F_c = jumlah filter (*output channels*).

I_c = jumlah *input channels*.

F_k = tinggi filter (*kernel height*).

F_l = lebar filter (*kernel width*).

bias = jumlah bias (umumnya sama dengan F_c karena setiap filter biasanya memiliki satu bias).

$$\text{Ukuran akhir} = \frac{n + 2p - f}{s} + 1 \quad (2.3)$$

n = ukuran dimensi *input*

p = *padding*, penambahan piksel di sekitar tepi *input*

f = ukuran filter

s = *stride*, pergeseran / langkah filter berpindah setiap operasi konvolusi

Maka, contoh pada Gambar 2.7 akan memiliki total parameter dan ukuran dimensi akhir adalah sebagai berikut

$$\begin{aligned} \text{Jumlah parameter} &= 1 \times 2 (2 \times 2) + 1 \\ &= 2 \times 4 + 1 \\ &= 8 + 1 \\ &= 9 \text{ parameter} \end{aligned}$$

$$\begin{aligned} \text{Ukuran akhir FM1} &= \frac{3+0-2}{1} + 1 \\ &= 2, \text{ maka ukuran terakhir } 2 \times 2 \end{aligned}$$

2.3.2. Fungsi Aktivasi

Fitur-fitur data citra yang telah diekstrak saat proses konvolusi perlu diolah lebih lanjut sebelum memasuki tahap *pooling*. Pada tahap ini, data citra diproses menggunakan fungsi aktivasi, yaitu sebuah fungsi non-linear yang berperan penting dalam meningkatkan kemampuan representasi model CNN. Fungsi aktivasi memungkinkan model untuk mempelajari non-linearitas, sehingga mampu mempelajari pola yang lebih kompleks dan tidak terduga dalam citra. Penelitian ini menggunakan dua jenis aktivasi pada model CNN, yaitu fungsi aktivasi ReLU dan fungsi aktivasi *softmax*. Berikut adalah penjelasan dari fungsi aktivasi tersebut:

a. Fungsi *Rectified Linear Unit* (ReLU)

Fungsi aktivasi *Rectified Linear Unit* (ReLU) memetakan *input* menjadi 0 jika nilai tersebut negatif, dan tetap mempertahankan nilai tersebut jika positif.

$$f_{ReLU} = \begin{cases} 0 & ; x \leq 0 \\ x & ; x > 0 \end{cases} \quad (2.4)$$

Fungsi aktivasi ini memiliki beberapa kelebihan, termasuk perhitungan yang efisien, pelatihan yang cepat, dan kemampuan untuk mengatasi masalah gradien menghilang yang umum terjadi pada fungsi aktivasi lain seperti *sigmoid* dan *tanh* (Bai, 2022). Kesederhanaan fungsi aktivasi ReLU mudah untuk diimplementasikan dan diinterpretasikan, sehingga mengurangi kompleksitas model dan membuatnya lebih cepat untuk diproses.

Input					Output			
12	33	24	54	→ Relu	12	33	24	54
3	23	-11	51		3	23	0	51
-1	-8	32	28		0	0	32	28
45	34	-20	-18		45	34	0	0
-9	32	7	2		0	32	7	2

Gambar 2.9 Ilustrasi fungsi aktivasi ReLU

b. Fungsi *Softmax*

Fungsi aktivasi *softmax* memetakan vektor *input* ke dalam vektor probabilitas, di mana setiap elemen dalam vektor probabilitas merepresentasikan probabilitas dari kelas tertentu. Fungsi ini digunakan pada *layer* akhir model CNN sehingga menghasilkan klasifikasi akhir dalam bentuk probabilitas.

$$\begin{aligned} \text{Softmax}(\vec{y})_i \\ = \frac{e^{\vec{y}_i}}{\sum_{j=1}^K e^{\vec{y}_j}} \end{aligned} \quad (2.5)$$

Nilai-nilai dalam vektor probabilitas yang dihasilkan dari fungsi aktivasi *softmax*, selalu bernilai antara 0 dan 1. Jumlah semua nilai vektor akan selalu sama dengan 1, nilai probabilitas tertinggi dalam vektor menunjukkan kelas yang paling mungkin untuk *input* yang telah diberikan.

$$y = \begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \end{bmatrix} \longrightarrow \text{Softmax}(\vec{y})_i = \frac{e^{\vec{y}_i}}{\sum_{j=1}^3 e^{\vec{y}_j}} \longrightarrow P_y = \begin{bmatrix} 0.659 \\ 0.242 \\ 0.099 \end{bmatrix}$$

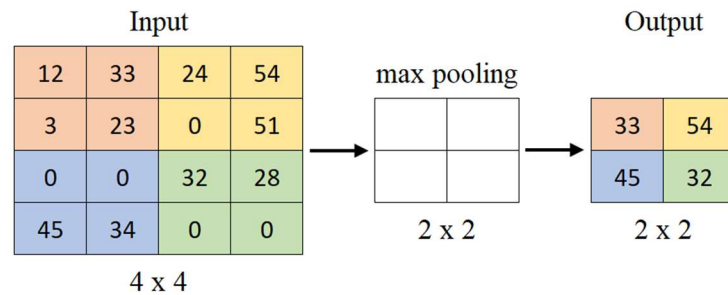
Gambar 2.10 Ilustrasi fungsi aktivasi softmax

y merupakan lapisan neuron terakhir dengan 3 *output*. Fungsi *softmax* akan menghasilkan vektor probabilitas berdasarkan y . Nilai dengan probabilitas tertinggi yaitu 0.659 menunjukkan bahwa elemen pertama memiliki probabilitas tertinggi sebagai kelas yang benar.

2.3.3. *Pooling Layer*

Pooling layer merupakan salah satu komponen penting dalam arsitektur CNN. *Pooling layer* adalah langkah yang penting dalam model CNN untuk mengurangi dimensi spasial dari *feature map*, namun juga tetap mempertahankan informasi penting dari data *input* (Gholamalinezhad & Khosravi, 2020). Letak *pooling layer* pada umumnya berada setelah *layer* konvolusi. *Pooling layer* yang digunakan dalam penelitian ini yaitu *max pooling*.

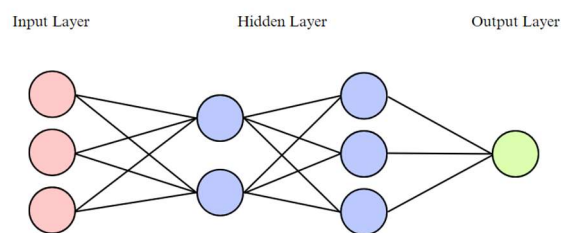
Max pooling merupakan salah satu jenis *pooling layer* yang memilih nilai maksimum dari dalam *kernel pooling* untuk menghasilkan nilai *output*. *Kernel pooling* merupakan area kecil yang bergerak ke seluruh *feature map* untuk melakukan operasi *pooling*. Ukuran *kernel pooling* dan Langkah (*stride*) yang digunakan akan mempengaruhi hasil *output*.



Gambar 2.11 Ilustrasi *pooling layer*

2.3.4. Fully Connected Layer

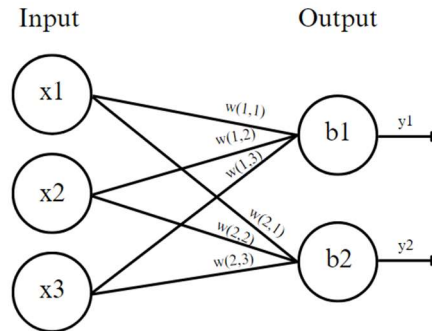
Fully connected layer, disebut juga dengan *dense layer*, merupakan komponen penting dalam arsitektur model CNN. *Layer* ini berada setelah lapisan konvolusi dan bagian akhir dari arsitektur CNN, bertanggung jawab dalam melakukan klasifikasi berdasarkan fitur-fitur yang telah diekstraksi sebelumnya oleh *layer* konvolusi.



Gambar 2.12 Struktur Dasar *Dense Layer*

Struktur dasar dari *dense layer* terdiri dari tiga komponen utama, yaitu *input*, *hidden layer*, dan *output*. Ketiga struktur ini saling terhubung satu sama lain membangun jaringan seperti saraf manusia. *input layer* merupakan lapisan pertama yang menerima data *input* dari *layer* sebelumnya untuk dilanjutkan menuju *hidden layer*. *Hidden layer* merupakan lapisan tersembunyi yang memproses data dari *input layer*, dan meneruskannya ke *output layer*. *Hidden layer*

terletak di antara *input layer* dan *output layer*. *output layer* merupakan *layer* terakhir yang memberikan hasil akhir dalam model CNN.



Gambar 2.13 Unsur Dasar Parameter CNN

Unsur dasar dari CNN yang berfungsi sebagai parameter model yaitu *weight* dan bias. Keduanya merupakan komponen penting yang akan dioptimalkan selama proses latih model untuk memaksimalkan performa model dalam melakukan tugas-tugas seperti klasifikasi gambar. *Weight* merupakan parameter yang menentukan bagaimana setiap *input* yang dikombinasikan menghasilkan *output* di setiap neuron. Parameter ini memiliki nilai yang berbeda setiap neuron asal yang menuju neuron selanjutnya. Bias merupakan faktor lain yang terkait dengan penyimpanan informasi. Bias dalam bentuk nilai konstan ini ditambahkan pada *input* setiap neuron dengan tujuan menentukan aktivitas awal neuron. Setiap layer memiliki parameter sesuai dengan jumlah neuron, dan setiap neuron melakukan operasi linear terhadap *input*, berikut secara matematis *output* dan total parameter dari neuron j adalah:

$$y_j = \sum_{i=1}^n W_{i,j} \cdot x_i + b_j \quad (2.6)$$

$$\text{Total parameter} = n_a \times (n_{a-1} + 1)$$

x_i adalah *input*, $w_{i,j}$ adalah bobot (*weight*) yang menghubungkan neuron *input* ke neuron selanjutnya, n_a adalah jumlah neuron layer saat ini, dan n_{a-1} adalah jumlah neuron layer sebelumnya. Bobot

memiliki nilai yang berbeda-beda setiap *input* dan neuron yang dituju. Berikut adalah unsur dasar berdasarkan Gambar 2.13:

1. *Input* vektor (x):

$$x = [x_1, x_2, x_3]$$

2. *Weight* (W)

$$W = \begin{bmatrix} w_{1,1} & w_{2,1} & w_{3,1} \\ w_{1,2} & w_{2,2} & w_{3,2} \end{bmatrix}$$

3. Bias (b)

$$b = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

Berdasarkan parameter yang ada, didapatkan total parameter fungsi yang terjadi adalah sebagai berikut :

$$y_j = \sum_{j=1}^3 w_{i,j} \cdot x_j + b_i \quad (2.7)$$

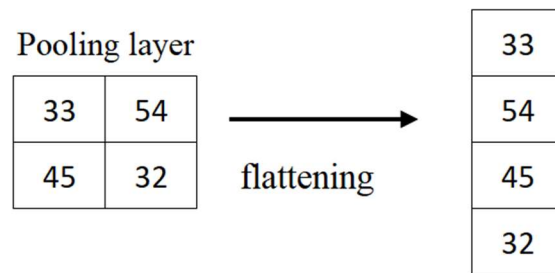
$$y_j = \begin{bmatrix} w_{1,1} & w_{2,1} & w_{3,1} \\ w_{1,2} & w_{2,2} & w_{3,2} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} (w_{1,1} \times x_1) + (w_{1,2} \times x_2) + (w_{1,3} \times x_3) + b_1 \\ (w_{2,1} \times x_1) + (w_{2,2} \times x_2) + (w_{2,3} \times x_3) + b_2 \end{bmatrix}$$

$$\begin{aligned} \text{Total Parameter} &= n_a \times (n_{a-1} + 1) \\ &= 2 \times (3 + 1) \\ &= 8 \text{ Parameter} \end{aligned}$$

2.3.5. *Flattening*

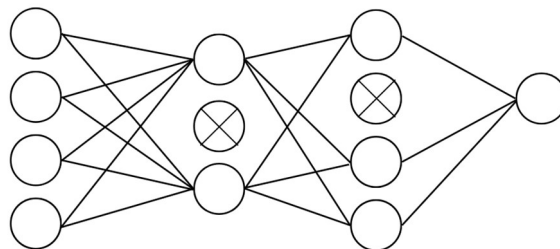
Flattening merupakan tahap yang sederhana namun penting dalam arsitektur CNN yang terletak setelah *Pooling layer*. Tahap ini bekerja dengan mengubah representasi spasial dari fitur yang telah diekstraksi menjadi vektor 1 dimensi yang kemudian diproses *dense layer*. Tahap ini bertujuan untuk meningkatkan efisiensi komputasi.



Gambar 2.14 Proses *Flattening*

2.3.6. *Dropout*

Dropout merupakan salah satu teknik regularisasi yang sering digunakan dengan tujuan mencegah terjadinya model *overfitting*. *Overfitting* merupakan suatu kondisi di mana model terlalu mempelajari informasi pada data latih sehingga model tidak mampu memprediksi data baru secara akurat. Pada metode ini, neuron akan secara acak dipilih untuk dihilangkan selama proses pelatihan model. Jumlah neuron yang dihilangkan dapat diatur, jika *dropout rate* bernilai 0.5 maka 50% neuron pada *layer* tersebut akan dihilangkan.



Gambar 2.15 Ilustrasi Proses *Dropout*

2.3.7. Citra

Menurut Kirana (2021), citra merupakan imitasi benda tiga dimensi yang direpresentasikan dalam bentuk dua dimensi melalui kombinasi garis titik, bentuk, dan warna. Citra berdasarkan teknologi digital adalah sebuah kombinasi piksel yang terdiri atas warna, sehingga dapat membentuk imitasi objek dan tekstur tertentu. Informasi mendasar yang didapat tersebut akan dilakukan analisa dan dikenali oleh komputer untuk mengidentifikasi situasi tertentu. Dalam proses analisis informasi, diperlukan pemilihan informasi yang akan diekstrak berdasarkan permasalahan yang sedang dianalisis.

Citra dibagi menjadi dua berdasarkan sifat nilai dari intensitas warna, yaitu citra digital dan analog (Kirana, 2021). Citra digital adalah kombinasi dari piksel yang terdiri atas nilai diskrit, sedangkan citra analog adalah kombinasi dari piksel yang terdiri atas nilai kontinu. Penerapan citra digital salah satunya ada pada aplikasi pertanian, yaitu dengan memanfaatkan citra digital dalam perdagangan dan pertanian dapat melakukan sortir kualitas dan klasifikasi kualitas hasil.

2.3.8. *Confusion Matrix*

Tabel 2.1 *Confusion Matrix* Klasifikasi

		Hasil Prediksi	
Hasil Sebenarnya	Kelas	Kelas 1	Kelas 2
	Kelas 1	<i>True Positive</i> (TP)	<i>False Negative</i> (FN)
	Kelas 2	<i>False Positive</i> (FP)	<i>True Negative</i> (TN)

Confusion matrix merupakan suatu metode yang digunakan untuk melakukan evaluasi dengan menghitung tingkat kebenaran dari proses klasifikasi. Pada metode evaluasi ini, disajikan dalam format tabel yang menunjukkan jumlah prediksi yang benar dan salah pada setiap kelas atau label. *Confusion Matrix* dibagi menjadi dua jenis, yaitu *confusion matrix biner* sesuai dengan namanya untuk klasifikasi dua kelas, dan *confusion matrix multiclass* untuk klasifikasi lebih dari dua kelas. Untuk klasifikasi lebih dari dua kelas dilakukan evaluasi hasil dengan menentukan metrik evaluasi pada setiap label, hal ini dilakukan agar memperoleh gambaran lebih lengkap mengenai performa model klasifikasi.

Pada Tabel 2.1, *True positive* menerangkan jumlah data yang terdapat pada kelas 1 diprediksi dengan benar, *False Negative* menerangkan jumlah data yang terdapat pada kelas 1 namun salah

diprediksi sebagai kelas 2. *False Positive* menerangkan jumlah data yang terdapat pada kelas 2 diprediksi dengan salah sebagai kelas 1, sedangkan *True Negative* menerangkan jumlah data pada kelas 2 benar memprediksi dengan hasil kelas 2. Nilai-nilai yang didapat tersebut dijadikan sebagai acuan dalam menghitung beberapa indikator dalam melakukan evaluasi sistem, yaitu akurasi, presisi, dan *recall*.

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.8)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.9)$$

$$\text{Presisi} = \frac{TP}{TP + FP} \quad (2.10)$$

$$\text{F1 score} = \frac{2(\text{Presisi} \times \text{Recall})}{\text{Presisi} + \text{Recall}} \quad (2.11)$$

BAB III METODOLOGI PENELITIAN

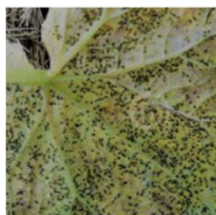
3.1. Jenis Penelitian

Penelitian ini merupakan penelitian terapan yang berfokus pada penerapan pengetahuan dan teori untuk mengatasi permasalahan praktis dengan tujuan terukur berupa pengembangan model *Convolutional Neural Network* (CNN) yang tepat dan efisien. Metode yang digunakan dalam penelitian ini adalah dengan menerapkan dan mengevaluasi model CNN untuk mengklasifikasikan jenis penyakit dan hama tanaman kapas.

3.2. Variabel dan Parameter Penelitian

Penelitian ini menggunakan enam variabel, berikut adalah jenis-jenis penyakit dan hama daun tanaman kapas yang digunakan dalam penelitian ini:

Tabel 3.1 Variabel Penelitian

Penyakit		
<i>Bacterial Blight</i>	<i>Powdery Mildew</i>	<i>Target Spot</i>
		
Hama		
<i>Aphid</i>	<i>Army Worm</i>	
		
Tanpa Penyakit dan Hama		
<i>Healthy</i>		
		

Parameter yang digunakan dalam penelitian ini antara lain bobot (*weight*), dan bias. Parameter ini tidak bisa diobservasi secara langsung karena sudah ditentukan secara komputasi. Berikut adalah definisi parameter yang digunakan dalam penelitian ini disajikan dalam bentuk tabel.

Tabel 3.2 Daftar parameter model CNN klasifikasi tanaman kapas

No	Parameter	Definisi
1	weight	satuan nilai yang diberikan pada masing-masing koneksi antara neuron-neuron di dalam jaringan saraf tiruan.
2	bias	Nilai konstan yang ditambahkan ke hasil dari perhitungan jaringan saraf setelah semua <i>input</i> telah dikalikan dengan <i>weight</i> masing-masing

Selain parameter, penelitian ini juga melibatkan *hyperparameter*. *Hyperparameter* adalah variabel yang nilainya ditentukan sebelum proses pelatihan dan digunakan untuk mengontrol proses pembelajaran model. Berikut adalah beberapa *hyperparameter* yang digunakan dalam penelitian ini.

Tabel 3.3 Daftar *hyperparameter* model CNN klasifikasi tanaman kapas

<i>Hyperparameter</i>	Keterangan
Ukuran kernel	Dimensi kernel konvolusi
Jumlah channel (<i>Depth</i>)	Jumlah filter yang digunakan
Jumlah <i>feature map</i>	Hasil <i>feature map</i> yang dihasilkan
<i>Strides</i>	Pergeseran filter pada data input
<i>Padding</i>	Penambahan 0 pada sekitar output
<i>Batch size</i>	Ukuran sampel yang diproses sebelum model diperbarui

<i>Hyperparameter</i>	Keterangan
<i>Epoch</i>	Jumlah iterasi melalui data selama pelatihan model
<i>Fungsi Aktivasi</i>	Fungsi aktivasi yang digunakan pada <i>layer</i> tertentu

3.3. Jenis dan Sumber Data

Data yang digunakan dalam penelitian ini berupa data sekunder, yaitu data yang didapatkan melalui situs *Kaggle* pada tahun 2023. Data ini berasal dari Dhamodharan, keseluruhan data berjumlah 3601 citra jenis penyakit dan hama daun tanaman kapas. Data ini terbagi menjadi enam kelas, yaitu *Bacterial Blight*, *Powdery Mildew*, *Target spot*, *Aphids*, *Army Worm*, dan sehat. Data dalam penelitian ini dibagi menjadi tiga set, yaitu :

- Data latih (75%), data ini digunakan untuk melatih model CNN. Data latih ini terdiri dari 2700 citra daun kapas dengan masing-masing kelas terdiri atas 450 citra.
- Data validasi (15%), digunakan untuk mengevaluasi performa model selama proses pelatihan model berlangsung. Data validasi ini terdiri dari 540 citra daun kapas dengan masing-masing kelas terdiri atas 90 citra.
- Data tes (10%), digunakan untuk mengevaluasi performa model akhir setelah pelatihan selesai. Data tes terdiri dari 361 citra daun kapas dengan masing-masing kelas terdiri atas 60 citra, kecuali kelas *target spot* berjumlah 61 citra.

3.4. Teknik Analisis Data

Teknik analisis data yang digunakan dalam penelitian ini adalah metode *Convolutional Neural Network* (CNN), yaitu salah satu dari metode *Deep Learning* yang efektif digunakan untuk melakukan pengenalan pola pada data gambar dengan baik.

Langkah analisis dalam penelitian yang dilakukan untuk mendapatkan model CNN adalah sebagai berikut:

1. Membagi data menjadi tiga bagian, yaitu data latih, data validasi, dan data tes, dengan rasio pembagian masing-masing 75%;15%;10%.
2. Melakukan data *preprocessing* dan augmentasi agar model mampu menghasilkan performa yang baik. Data *preprocessing* dilakukan untuk memastikan data memiliki format dan karakteristik yang seragam, sehingga mempercepat proses pelatihan model.

- a. *Resize* data

Sebelum digunakan dalam proses latih, data diubah ukurannya menjadi 120x120 piksel. Penggunaan ukuran ini berdasarkan pada keseimbangan antara mempertahankan informasi penting dan efisiensi komputasi. Jika ukuran gambar terlalu kecil, informasi yang diproses akan terlalu sedikit, dan jika terlalu besar maka proses latih model akan lebih lama karena banyak data yang diproses

- b. Normalisasi data

Setelah pengubahan ukuran, tahap selanjutnya adalah normalisasi data. Normalisasi penting dilakukan untuk memastikan bahwa semua fitur berada dalam rentang yang sama, sehingga dapat meningkatkan performa dan stabilitas model selama pelatihan. Karena data dalam bentuk gambar memiliki nilai intensitas piksel maksimum sebesar 255, maka setiap nilai piksel akan dibagi dengan 255. Proses ini akan mengubah nilai piksel ke dalam rentang 0 hingga 1.

Melakukan data augmentasi memungkinkan untuk meningkatkan jumlah dan variasi data latih tanpa perlu mengumpulkan data tambahan secara manual. Data augmentasi bermanfaat untuk meningkatkan performa model klasifikasi karena membantu model belajar lebih banyak variasi data

3. Membangun arsitektur model CNN

Setelah data diproses, selanjutnya adalah membangun arsitektur model CNN. Arsitektur ini terbagi menjadi 2 bagian utama, yaitu tahap ekstraksi fitur dan tahap klasifikasi. Tahap ekstraksi fitur terdiri dari

beberapa lapisan, yaitu lapisan konvolusi dan lapisan *pooling*. Tahap klasifikasi terdiri atas *fully connected layer*, lapisan ini memungkinkan model untuk menggabungkan semua fitur yang telah diekstraksi sebelumnya dan membuat keputusan akhir terkait klasifikasi gambar.

4. Melatih model CNN

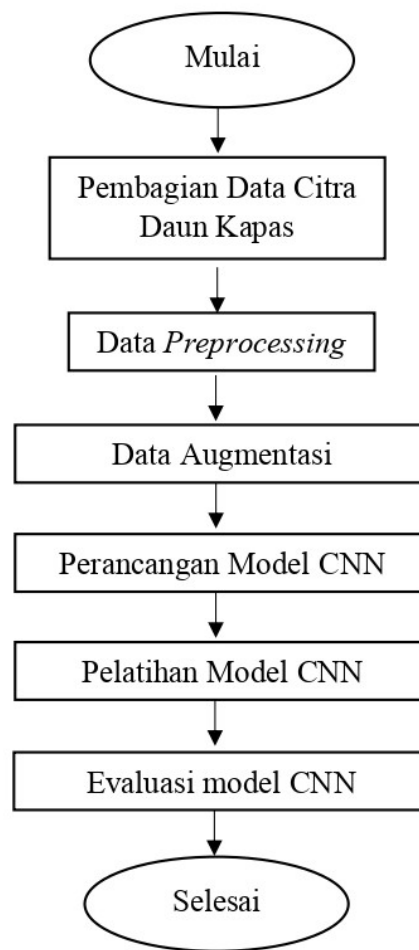
Pada tahap ini, model CNN menggunakan data latih dan data validasi yang telah dilakukan data *preprocessing* dan data augmentasi. Model dilatih menggunakan sejumlah *batch* data yang ditentukan dan akan mengulang prosesnya dengan *batch* yang berbeda. Pada setiap iterasi, model memperbaiki parameter setiap *layer* agar dapat meminimalkan *loss* dan meningkatkan akurasi.

5. Evaluasi Model CNN

Setelah model dilatih, performa model perlu dievaluasi dengan menggunakan data tes yang belum pernah dilihat sebelumnya untuk memastikan generalisasi model terhadap data baru. Evaluasi ini dilakukan dengan menghitung akurasi serta memanfaatkan *confusion matrix* pada setiap label.

3.5. Diagram Alir Penelitian

Tahapan selama penelitian dilakukan dalam beberapa tahap seperti pada Gambar 3.1,



Gambar 3.1 Diagram Alir Tahapan Penelitian

BAB IV HASIL DAN PEMBAHASAN

4.1. Hasil Analisis Data

Penelitian ini diawali dengan proses pengumpulan data, data yang digunakan dalam penelitian ini adalah data citra penyakit dan hama daun tanaman kapas. Sumber data yang didapatkan melalui platform *Kaggle* yang berasal dari akun Dhamodharan. Proses pengumpulan data dilakukan dengan mencari data yang sesuai melalui *kaggle*, pencarian dihentikan saat data yang terkumpul sudah cukup.

Data yang dikumpulkan akan dibagi menjadi tiga bagian. Pembagian data menjadi data latih, data validasi, dan data tes dengan masing-masing memiliki rasio pembagian 75%;15%;10%. Data latih digunakan untuk melatih model dengan data validasi untuk mengevaluasi model selama proses latih. Sedangkan data tes digunakan untuk mengevaluasi akhir model yang telah dilatih, data ini merupakan data yang belum pernah dilihat saat proses pelatihan.

Data selanjutnya akan melalui proses data *processing*, proses ini meliputi *rescale* dan *resize*. *Rescale* mengubah ukuran menjadi seragam, ukuran yang ditentukan dalam penelitian ini yaitu 120x120 piksel. Selanjutnya adalah *rescale*, yaitu mengubah nilai setiap piksel dalam gambar menjadi dalam rentang 0-1 piksel. Dilakukan skala ulang ini untuk memastikan konsistensi data sehingga model bisa belajar dengan cepat.

Selanjutnya dilakukan data augmentasi, proses ini dilakukan dengan tujuan menambahkan variasi data baru yang berbeda. Data augmentasi yang dilakukan pada penelitian ini dapat dilihat dalam tabel ini :

Tabel 4.1 Daftar Teknik Data Augmentasi

No.	Augmentasi	
1.	<i>Shear</i>	
	Memiringkan gambar	

No.	Augmentasi	
2.	<i>Rotation</i>	
	Rotasi pada gambar 20 derajat	
3.	<i>Zoom</i>	
	Membesarkan gambar sebanyak 0.2	
4.	<i>Horizontal flip</i>	
	Melakukan pembalikan secara horizontal	

4.2. Perancangan Model CNN

Model: "sequential"

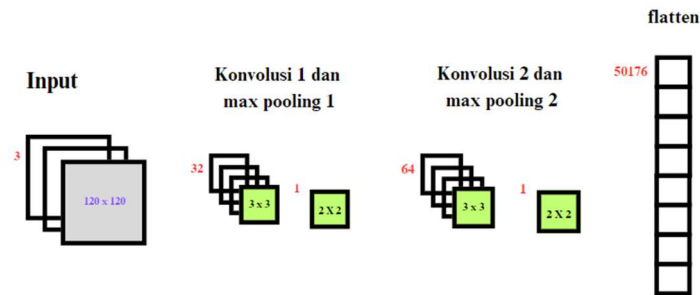
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 118, 118, 32)	896
max_pooling2d (MaxPooling2D)	(None, 59, 59, 32)	0
conv2d_1 (Conv2D)	(None, 57, 57, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 28, 28, 64)	0
flatten (Flatten)	(None, 50176)	0
dropout (Dropout)	(None, 50176)	0
dense (Dense)	(None, 256)	12845312
dropout_1 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32896
dense_2 (Dense)	(None, 6)	774

Gambar 4.1 Ringkasan Model Jaringan

Perancangan model terbagi menjadi dua, yaitu ekstraksi fitur dan klasifikasi. Ekstraksi fitur merupakan *layer* yang digunakan model untuk mempelajari cir-ciri penting yang terdapat dalam data. Tahap klasifikasi kemudian memanfaatkan hasil data dari ekstraksi fitur untuk melakukan

kategori data ke dalam kelas yang sesuai. Gambar 4.1 memberikan gambaran menyeluruh tentang model jaringan saraf yang telah dibuat. Gambar ini membantu memahami jumlah lapisan dan kompleksitas model secara keseluruhan. Berikut adalah penjelasan rinci mengenai setiap lapisan model yang dibangun.

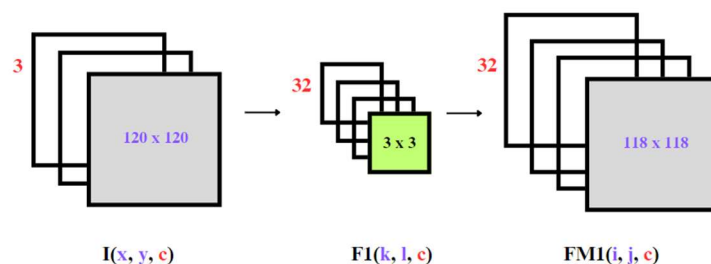
4.2.1. Tahap Ekstraksi fitur



Gambar 4.2 Tahap Ekstraksi fitur

Tahap ekstraksi fitur merupakan tahap dimana model memahami fitur yang terdapat dalam data. Pada tahap ini, model memproses data melalui serangkaian *layer* yang terdiri dari operasi konvolusi dan *Pooling*.

a. *Convolution Layer 1 (C1)*



Gambar 4.3 Convolution *Layer 1 (C1)*

Convolution Layer 1 (C1) merupakan *layer* pertama dalam tahap pembelajaran fitur yang berperan untuk memfilter data *input*. Filter ini berukuran 3x3 dengan jumlah *channel* sebanyak 32, bertugas mengekstrak fitur-fitur penting dari data *input* yang berukuran 120x120 dengan tiga *channel* warna (RGB). Operasi konvolusi pada *layer C1* menghasilkan 32 *feature map* dengan

dimensi 118x118 piksel. *Hyperparameter* pada *layer* C1 dapat dilihat pada table berikut :

Tabel 4.2 Hyperparameter *layer* C1

<i>Convolution Layer 1 (C1)</i>		
Hyperparameter	Nilai	Deskripsi
Ukuran <i>kernel</i>	3x3	Dimensi <i>kernel</i> konvolusi
Jumlah channel (<i>Depth</i>)	32	Jumlah filter yang digunakan
Jumlah <i>feature map</i>	32	Jumlah peta fitur yang dihasilkan
<i>Stride</i>	1	Pergeseran filter pada data <i>input</i>
<i>Padding</i>	0	Penambahan 0 pada sekitar <i>output</i>

Detail parameter C1 dapat dilihat pada **Tabel 4.3**. Tabel ini menunjukkan dua komponen utama parameter *layer*:

Tabel 4.3 Parameter *layer* C1

<i>Convolution Layer 1 (C1)</i>		
Parameter	Nilai	Deskripsi
Bias	32	Nilai konstan yang ditambahkan ke <i>output</i> dari setiap filter
Bobot	3x3x32	Nilai-nilai yang menentukan filter memproses <i>output</i>

Total parameter yang dimiliki oleh *layer* C1 berdasarkan tabel sebelumnya, dapat dirumuskan sebagai berikut :

$$\begin{aligned}
 \text{Total parameter} &= F1_c \times I_c (F_k \times F_l) + \text{bias} \\
 &= 32 \times 3 (3 \times 3) + 32 \\
 &= 32 \times 27 + 32 \\
 &= 864 + 32 \\
 &= \mathbf{896} \text{ parameter}
 \end{aligned}$$

$$\text{Ukuran akhir } FM1 = \frac{n+2p-f}{s} + 1$$

$$= \frac{120+0-3}{1} + 1$$

= 118, maka ukuran terakhir **118 x 118**

Fungsi aktivasi yang digunakan dalam *layer* C1 ini yaitu *Relu*. Hal ini memungkinkan hasil konvolusi untuk melewati nilai nol dan menghasilkan nilai positif. Fungsi ini membantu model untuk fokus pada fitur-fitur yang lebih penting dan mengabaikan nilai-nilai kecil yang tidak relevan. Pada *layer* C1, didapatkan persamaan matematika yang terjadi pada lapisan ini yaitu:

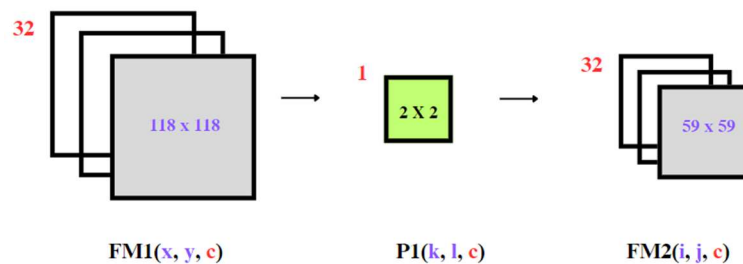
$$ReLU \left(\sum_{c=0}^2 \sum_{m=0}^2 \sum_{n=0}^2 I_{(i+m,j+n,c)} \times C_{1(m,n,c)} + bias \right)$$

(m,n) = ukuran kolom dan baris *kernel* C_1

(i, j) = ukuran kolom dan baris FM_1

c = *depth* atau kedalaman data

b. *Pooling Layer* 1 (Pool 1)

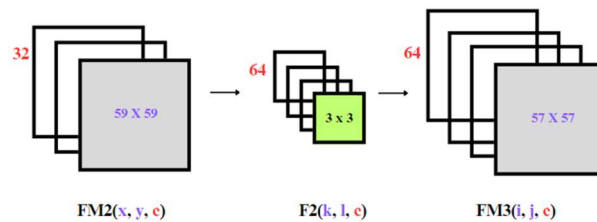


Gambar 4.4 *Pooling Layer* 1 (Pool 1)

Pooling Layer 1 (Pool 1) melanjutkan proses ekstraksi fitur penting dari data yang telah dilakukan *layer* sebelumnya. Pada *layer* ini, *input* yang berukuran 118×118 piksel akan diproses menggunakan operasi *max Pooling* dengan *kernel* ukuran 2×2 . Operasi ini memilih nilai tertinggi di setiap jendela 2×2 pada *input*, menghasilkan *output* dengan dimensi yang lebih kecil, yaitu 59×59 piksel. Berikut adalah tabel hyperparameter yang terdapat pada *layer* ini:

Tabel 4.4 Hyperparameter *MaxPooling Layer 1* (Pool 1)

<i>MaxPooling Layer 1</i> (Pool 1)		
Hyperparameter	Nilai	Deskripsi
Ukuran <i>kernel</i>	2x2	Dimensi <i>kernel</i> konvolusi
<i>Stride</i>	2	Pergeseran filter pada data <i>input</i>
<i>Padding</i>	0	Penambahan 0 pada sekitar <i>output</i>

c. *Convolution Layer 2* (C2)**Gambar 4.5** *Convolution Layer 2* (C2)

Convolution Layer 2 (C2) merupakan *layer* ketiga dalam tahap pembelajaran fitur yang berperan untuk memfilter data *input*. Filter ini berukuran 3x3 dengan jumlah *channel* sebanyak 64, bertugas mengekstrak fitur-fitur penting dari data *input* yang berukuran 59x59 piksel dengan tiga *channel* warna (RGB). Operasi konvolusi pada *layer* C2 menghasilkan 64 *feature map* (FM3) dengan dimensi 57X57 piksel. *Hyperparameter* pada *layer* C2 dapat dilihat pada table berikut :

Tabel 4.5 Hyperparameter *layer* C2

<i>Convolution Layer 2</i> (C2)		
Hyperparameter	Nilai	Deskripsi
Ukuran filter	3x3	Dimensi <i>kernel</i> konvolusi
Jumlah channel (<i>Depth</i>)	64	Jumlah filter yang digunakan
Jumlah <i>feature map</i>	64	Jumlah peta fitur yang dihasilkan
<i>Stride</i>	1	Pergeseran filter pada data <i>input</i>
<i>Padding</i>	0	Penambahan 0 pada sekitar <i>output</i>

Detail parameter C2 dapat dilihat pada Tabel 4.6. Tabel ini menunjukkan dua komponen utama parameter *layer*:

Tabel 4.6 Parameter *layer* C2

<i>Convolution Layer 2 (C2)</i>		
Parameter	Nilai	Deskripsi
Bias	64	Nilai konstan yang ditambahkan ke <i>output</i> dari setiap filter
Bobot	3x3x64	Nilai-nilai yang menentukan filter memproses <i>output</i>

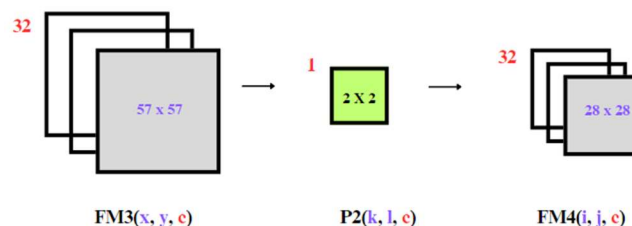
Total parameter yang dimiliki oleh *layer* C1 berdasarkan tabel sebelumnya, dapat dirumuskan sebagai berikut :

$$\begin{aligned}
 \text{Total parameter} &= F2_c \times FM2_c (F2_k \times F2_l) + bias \\
 &= 64 \times 32 (3 \times 3) + 64 \\
 &= 64 \times 288 + 64 \\
 &= 18432 + 64 \\
 &= \mathbf{18496} \text{ parameter}
 \end{aligned}$$

Fungsi aktivasi yang digunakan dalam *layer* C2 ini yaitu ReLU. Fungsi ini memungkinkan hasil konvolusi untuk melewati nilai nol dan menghasilkan nilai positif. Fungsi ini membantu model untuk fokus pada fitur-fitur yang lebih penting dan mengabaikan nilai-nilai kecil yang tidak relevan. Berikut adalah fungsi yang terjadi pada *layer* C2, yaitu:

$$ReLU \left(\sum_{c=0}^{31} \sum_{m=0}^2 \sum_{n=0}^2 I_{(i+m,j+r,c)} \times C_{1(m,n,c)} + bias \right)$$

d. *Pooling Layer 2 (Pool 2)*



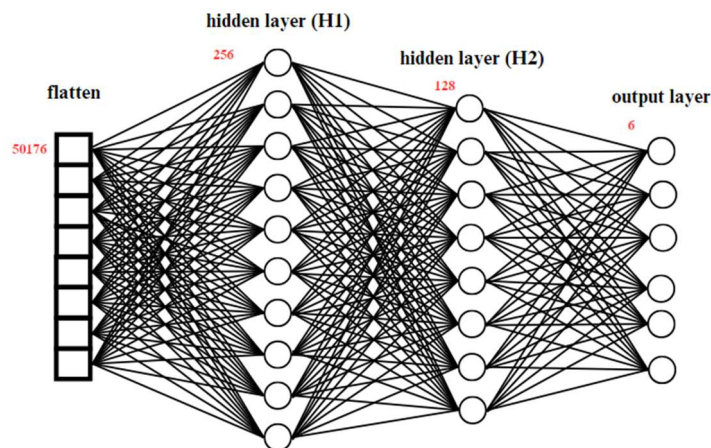
Gambar 4.6 *Pooling Layer 2 (Pool 2)*

Pooling Layer 2 (Pool 2) melanjutkan proses ekstraksi fitur penting dari data yang telah dilakukan *layer* sebelumnya. Pada *layer* ini, *input* yang berukuran 57x57 piksel akan diproses menggunakan operasi *max Pooling* dengan *kernel* ukuran 2x2. Operasi ini memilih nilai tertinggi di setiap jendela 2x2 pada *input*, menghasilkan *output* dengan dimensi yang lebih kecil, yaitu 28x28 piksel. Berikut adalah tabel parameter yang terdapat pada *layer* ini:

Tabel 4.7 Hyperparameter *MaxPooling Layer 2* (Pool 2)

<i>MaxPooling Layer 1</i> (Pool 1)		
Parameter	Nilai	Deskripsi
Ukuran <i>kernel</i>	2x2	Dimensi <i>kernel</i> konvolusi
<i>Stride</i>	2	Pergeseran filter pada data <i>input</i>
<i>Padding</i>	0	Penambahan 0 pada sekitar <i>output</i>

4.2.2. Tahap Klasifikasi



Gambar 4.7 Tahap Klasifikasi

Tahap klasifikasi bertanggung jawab dalam mengkategorikan data *input* berdasarkan fitur yang telah diekstraksi oleh lapisan konvolusi. Tahap ini melibatkan lapisan *fully connected layer*, yang berperan dalam menggabungkan fitur tersebut dan meletakkannya ke kategori *output* yang diinginkan. Setiap lapisan *fully connected* terhubung ke

setiap neuron di lapisan sebelumnya, sehingga memungkinkan model untuk menggabungkan fitur-fitur tersebut secara non-linear.

a. Hidden Layer 1 (H1)

Pada *layer* ini, hasil *flatten* dari proses *ekstraksi fitur* akan dilanjutkan ke *hidden layer 1* (H1). Setiap elemen yang terdapat pada *flatten* akan melalui setiap neuron yang ada pada H1 dengan *weight* yang berbeda-beda. *Flatten* yang memiliki dimensi 1 dan terdiri dari 50176 elemen, akan digunakan sebagai *input* pada H1. Berikut adalah unsur dari *layer* H1:

Input vektor *flatten* (x):

$$x = [x_1, x_2, x_3, \dots, x_{50176}]$$

Weight (W):

$$W = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,50176} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,50176} \\ \vdots & \vdots & \ddots & \vdots \\ w_{256,1} & w_{256,2} & \cdots & w_{256,50176} \end{bmatrix}$$

Bias (b):

$$b = [b_1, b_2, b_3, \dots, b_{256}]$$

Output (y):

$$y_j = \text{ReLU} \left(\sum_{i=1}^{50176} W_{j,i} \cdot x_i + b_j \right)$$

$$\begin{aligned} \text{Total Parameter H1} &= n_{H1} \times (n_{\text{flatten}} + 1) \\ &= 256 \times (50176 + 1) \\ &= 12845312 \text{ Parameter} \end{aligned}$$

Input merupakan vektor hasil dari *layer flatten* yang memiliki 50176 elemen. Total parameter yang dimiliki yaitu 12845312 parameter. *Input* yang memasuki setiap neuron akan memiliki *weight* yang berbeda, dan akan dijumlahkan dengan bias yang memiliki nilai konstan setiap neuronnya. Hasil yang didapatkan,

kemudian diaplikasikan fungsi ReLU untuk memperkenalkan non-linearitas ke dalam model.

b. Hidden Layer 2 (H2)

Pada *layer* ini, hasil *output* dari *hidden layer* 1 (H1) akan dilanjutkan ke *hidden layer* 2 (H2). Setiap elemen yang terdapat pada *output* H1 akan melalui setiap neuron pada H2 sebagai *input* dengan *weight* yang berbeda-beda. *input* ini berbentuk vektor dengan 256 elemen, berdasarkan hasil *output* pada *layer* H1 sebelumnya. Berikut adalah komponen-komponen dari *layer* H2:

Input vektor *flatten* (x):

$$x = [x_1, x_2, x_3, \dots, x_{256}]$$

Weight (W):

$$W = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,256} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,256} \\ \vdots & \vdots & \ddots & \vdots \\ w_{128,1} & w_{128,2} & \cdots & w_{128,256} \end{bmatrix}$$

Bias (b):

$$b = [b_1, b_2, b_3, \dots, b_{128}]$$

Output (y):

$$y_j = \text{ReLU} \left(\sum_{i=1}^{256} w_{j,i} \cdot x_i + b_j \right)$$

$$\text{Total Parameter H2} = n_{H2} \times (n_{H1} + 1)$$

$$= 128 \times (256 + 1)$$

$$= 32896 \text{ Parameter}$$

Input merupakan vektor hasil *output layer* H1 yang memiliki 256 elemen. Total parameter pada *layer* ini yaitu 32896 parameter. Setiap *input* yang memasuki setiap neuron akan memiliki *weight* yang berbeda, dan akan dijumlahkan dengan bias yang memiliki nilai konstan setiap neuronnya. Hasil yang didapatkan, kemudian diaplikasikan fungsi ReLU seperti yang terjadi pada *layer* sebelumnya.

c. *Output Layer (O)*

Layer ini merupakan *layer* terakhir yang akan dilalui, hasil *output* dari *hidden layer 2 (H2)* akan dilanjutkan ke *output layer*. Setiap elemen yang terdapat pada *output H1* akan melalui setiap neuron pada H2 sebagai *input* dengan *weight* yang berbeda-beda. *input* ini berbentuk vektor dengan 128 elemen, berdasarkan hasil *output* pada *layer H1* sebelumnya. Berikut adalah komponen-komponen dari *layer H2*:

Input vektor *flatten* (x):

$$x = [x_1, x_2, x_3, \dots, x_{128}]$$

Weight (W):

$$W = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,128} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,128} \\ w_{3,1} & w_{3,2} & \cdots & w_{3,128} \\ w_{4,1} & w_{4,2} & \cdots & w_{4,128} \\ w_{5,1} & w_{5,2} & \cdots & w_{5,128} \\ w_{6,1} & w_{6,2} & \cdots & w_{6,128} \end{bmatrix}$$

Bias (b):

$$b = [b_1, b_2, b_3, b_4, b_5, b_6]$$

Output (y):

$$y_j = \text{Softmax} \left(\sum_{i=1}^{128} w_{j,i} \cdot x_i + b_j \right)$$

$$\begin{aligned} \text{Total Parameter O} &= n_0 \times (n_{H2} + 1) \\ &= 6 \times (128 + 1) \\ &= 774 \text{ Parameter} \end{aligned}$$

Input merupakan vektor hasil *output layer H2* yang memiliki 128 elemen. Karena jumlah kelas yang akan diprediksi ada 6, maka jumlah neuron pada *output layer (O)* berjumlah 6 neuron. Total parameter yang dimiliki *layer* ini yaitu 774 parameter. Setiap *input* yang memasuki setiap neuron akan memiliki *weight* yang berbeda, dan akan dijumlahkan dengan bias yang memiliki nilai konstan setiap neuronnya. Hasil yang didapatkan akan diaplikasikan fungsi

aktivasi *Softmax* untuk mendapatkan probabilitas prediksi dari masing-masing kelas.

4.3. Hasil Latih Model

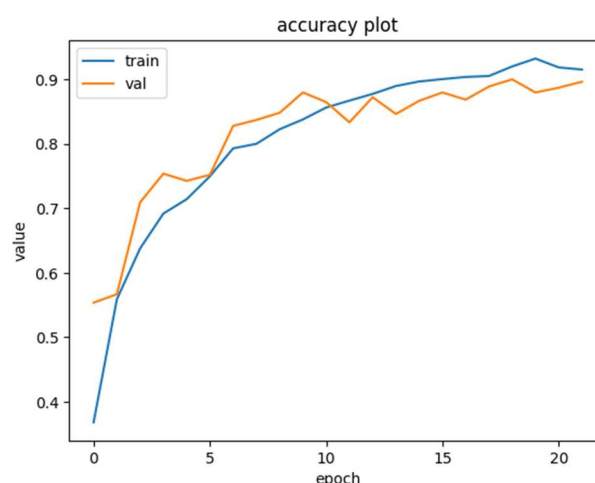
Setelah dirancang, model dilatih dengan menggunakan data latih dan data validasi. Data validasi digunakan untuk mengevaluasi performa model selama proses pelatihan. Proses latih ini berlangsung selama 22 iterasi dengan *batch size* 16, yang berarti seluruh data dilatih sebanyak 22 kali selama proses ini.

```
Epoch 22/25
169/169 [=====] - 129s 762ms/step - loss: 0.2308 - accuracy: 0.9152
```

```
recall: 0.9004 - val_loss: 0.3805 - val_accuracy: 0.8963
```

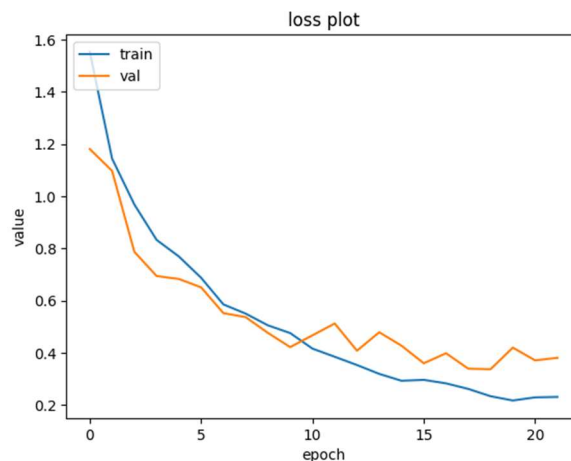
Gambar 4.8 Hasil akhir latih model

Berdasarkan Gambar 4.8, performa model dievaluasi menggunakan dua metrik utama, yaitu akurasi dan *recall*. Akurasi untuk mengukur proporsi prediksi model yang benar, dan *recall* untuk mengukur proporsi label positif yang benar-benar diidentifikasi oleh model. Model mencapai akurasi 91% pada data latih dan 89% pada data validasi, hal ini menunjukkan model mampu memprediksi kelas dengan baik pada kedua set data. *Recall* model pada data latih mencapai 90% dan 88% pada data validasi, hal ini menunjukkan kemampuan model mengklasifikasi label positif dengan baik.



Gambar 4.9 Akurasi Proses Pelatihan Model

Gambar 4.9 menunjukkan kurva akurasi selama proses pelatihan. Kurva menunjukkan akurasi model pada data latih dan validasi meningkat secara bertahap seiring dengan iterasi pelatihan, hal ini menunjukkan bahwa model mempelajari pola-pola dalam data dengan baik.



Gambar 4.10 Loss Proses Pelatihan Model

Pada Gambar 4.10, menunjukkan kurva *loss* model selama proses pelatihan. *Loss* merupakan nilai yang ingin diminimalkan selama proses pelatihan. Seiring dengan iterasi pelatihan, *loss* model pada data latih dan data validasi menurun secara bertahap. Hal ini menunjukkan model semakin optimal dalam melakukan prediksi kelas dengan benar.

4.4. Hasil Uji Model

Tahap ini dilakukan proses uji terhadap model yang telah dilatih sebelumnya menggunakan data tes daun kapas yang belum pernah dilihat sebelumnya. Data ini digunakan untuk mengevaluasi model akhir dengan total 361 citra daun kapas.

Tabel 4.8 Jumlah Data Tes Setiap Kelas

Kelas	Jumlah Data
<i>Aphids</i>	60
<i>Army Worm</i>	60
<i>Bacterial Blight</i>	60
<i>Healthy</i>	60
<i>Powdery Mildew</i>	60
<i>Target spot</i>	61

Uji coba ini menggunakan data yang telah dipisahkan sebelumnya. Data ini terdiri atas 6 *folder*, masing-masing diberi nama sesuai dengan labelnya. Jumlah data setiap label tercantum pada Tabel 4.8. Sebelum melakukan pengujian pada data baru, dilakukan *preprocessing*, yaitu data normalisasi dan diubahnya ukuran menjadi 120x120 piksel.

```

uploaded = files.upload()

for fn in uploaded.keys():
    path = fn
    img = image.load_img(path, target_size=size)
    imgplot = plt.imshow(img)

    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)
    x /= 255.0
    images = np.vstack([x])
    classes = model.predict(images, batch_size=10)[0]
    print(fn)
    print("Raw Predictions:", classes)

    sorted_indices = np.argsort(classes)[::-1]
    class_names = ['Aphids', 'Army Worm', 'Bacterial Blight', 'Healthy',
'Powdery Mildew', 'Target Spot']

    # Menampilkan kelas dengan probabilitas 3 terbesar
    top_classes = [(class_names[i], classes[i]*100) for i in sorted_indices[:3]]
    print("Top Predicted Classes:")

    for class_name, probability in top_classes:
        print(f'{class_name}: {probability:.2f}%')
    print(f'\n Result: {top_classes[0][0]}')
    print("Top Predicted Classes:")

```



```
for class_name, probability in top_classes:
    print(f'{class_name}: {probability:.2f}%')
```

Gambar 4.11 Kode pengujian klasifikasi penyakit dan hama

Proses klasifikasi dapat dilakukan dua cara, yaitu dengan memasukkan gambar secara individual yaitu satu per satu, dan pengujian klasifikasi secara otomatis. Memasukkan gambar secara manual proses ini melibatkan memasukkan gambar secara manual ke dalam model setelah menjalankan kode pada Gambar 4.11.

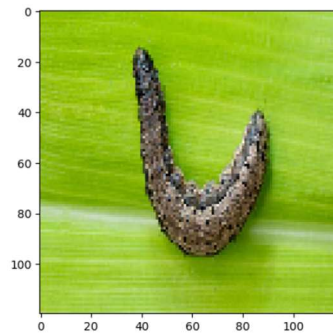


Gambar 4.12 Tampilan Uji Coba Untuk Klasifikasi

Menekan tombol *choose file* yang disediakan dapat memilih gambar dari penyimpanan lokal. Setelah gambar dipilih dan diunggah, gambar tersebut akan dikirim ke model untuk diproses dan diklasifikasikan secara otomatis. Hasil dari proses klasifikasi ini berupa probabilitas (peluang) untuk setiap kelas yang mungkin. Hasil tersebut diurutkan berdasarkan nilai probabilitasnya dari yang tertinggi ke terendah. Selanjutnya, model akan mengambil tiga kelas dengan probabilitas tertinggi sebagai hasil prediksi akhir. Hasil klasifikasi didapatkan berdasarkan kelas dengan probabilitas tertinggi.

Top Predicted Classes:
 Army Worm: 99.98%
 Aphids: 0.02%
 Healthy: 0.00%

Result: Army Worm



Gambar 4.13 Hasil Klasifikasi Benar Terklasifikasi *Army Worm*

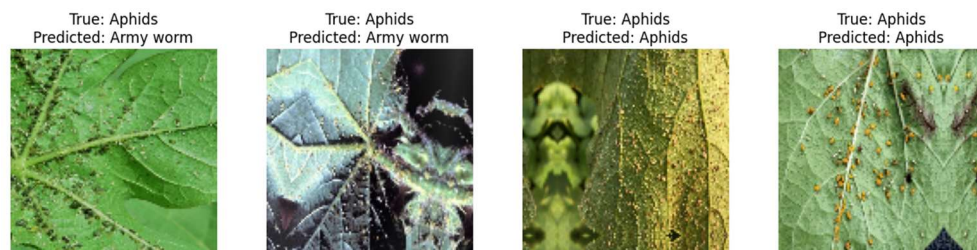
Untuk mengevaluasi kinerja model secara efisien, proses klasifikasi dilakukan secara otomatis. Hasil *preprocessing* yang dilakukan akan dimasukkan ke dalam model dan menghasilkan probabilitas setiap label, memungkinkan analisis performa model lebih mendalam.

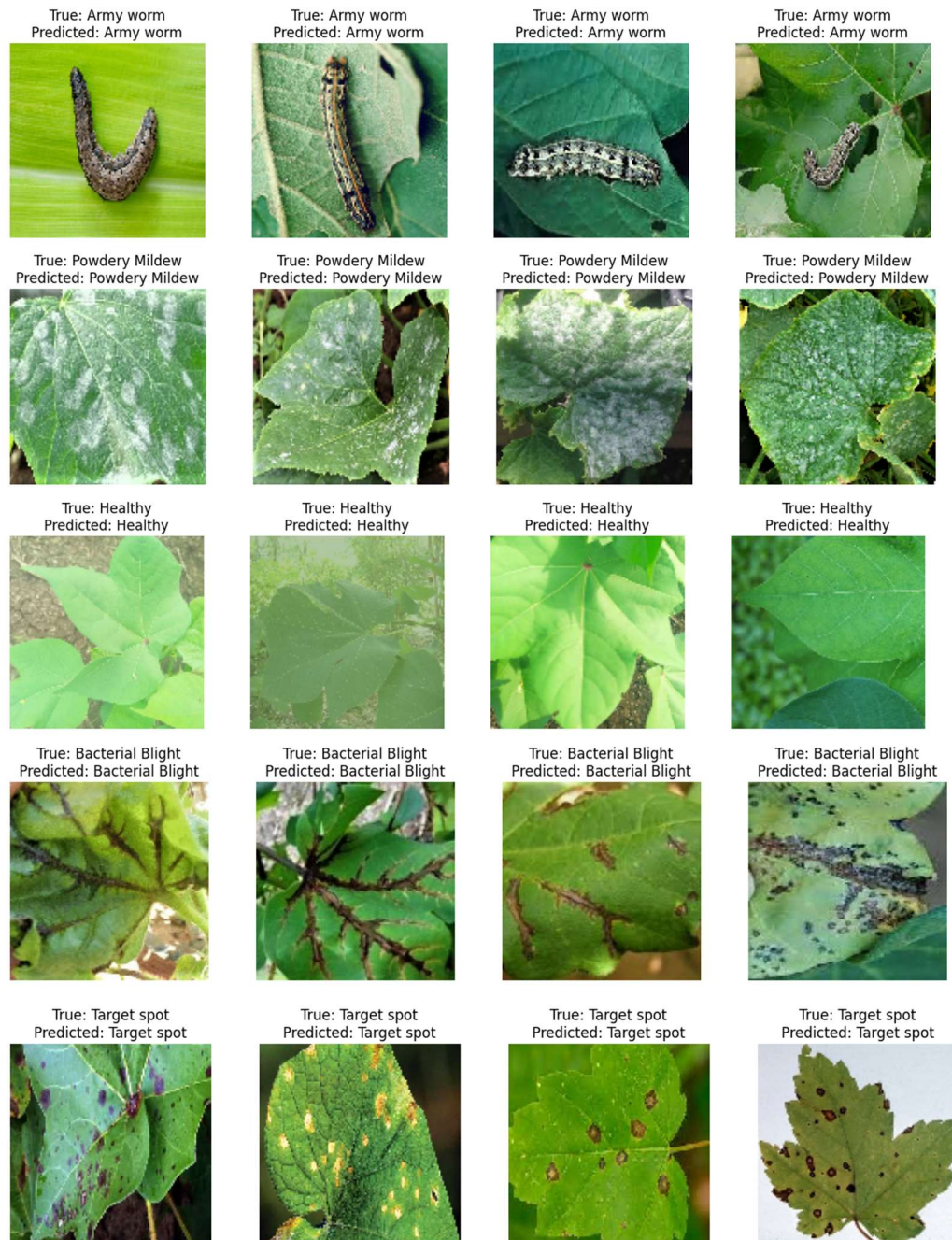
```
predictions = model.predict(test_generator)
```

23/23 [=====] - 7s 234ms/step

Gambar 4.14 Uji Coba Klasifikasi Menggunakan Data Tes

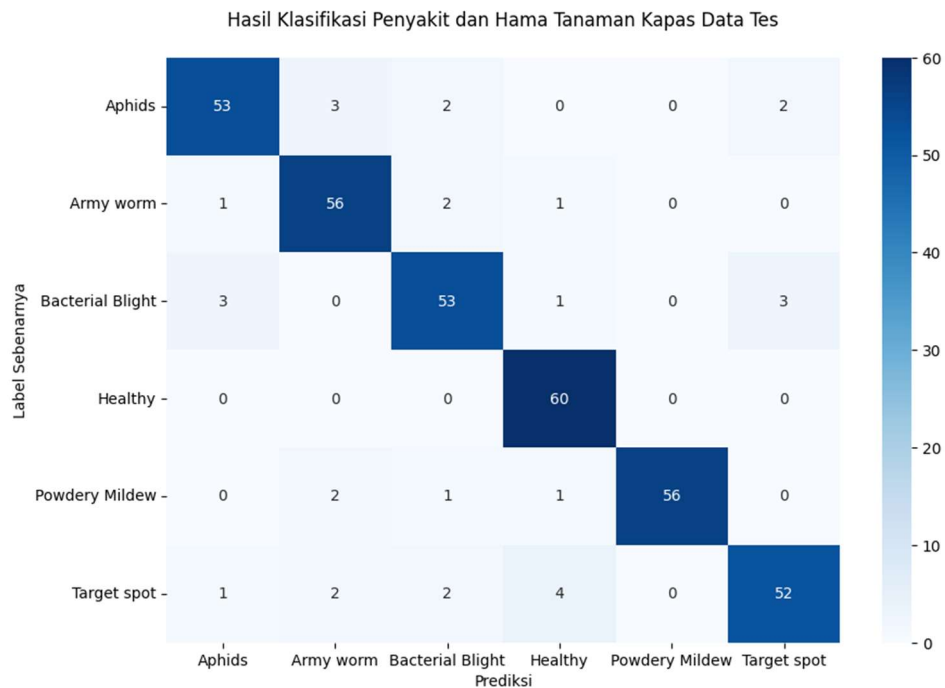
Setelah pengujian dilakukan, hasil klasifikasi akan disajikan dalam bentuk tabel sehingga memudahkan untuk pemahaman evaluasi model. Berikut adalah beberapa hasil prediksi dari data tes yang telah diklasifikasikan oleh model.





Gambar 4.15 Contoh beberapa sampel hasil klasifikasi

Hasil prediksi yang telah dihasilkan oleh model kemudian dibandingkan dengan label sebenarnya (*true labels*) untuk mengukur akurasi dan kesalahan klasifikasi. Gambar berikut ini menyajikan seluruh hasil prediksi model pada data tes, di mana kolom horizontal menunjukkan label yang sebenarnya, dan kolom vertikal menunjukkan hasil prediksi dari model.



Gambar 4.16 Hasil Prediksi Jenis Penyakit dan Hama

Berdasarkan gambar di atas, dapat dilihat hasil klasifikasi adalah sebagai berikut:

1. 58 citra *Aphids* terklasifikasi dengan benar, sedangkan 7 citra terklasifikasi salah yaitu 3 terklasifikasi *army worm*, 2 terklasifikasi *healthy*, dan 2 terklasifikasi *target spot*.
2. Terdapat 56 citra *army worm* terklasifikasi dengan benar, namun masih terdapat kesalahan klasifikasi sebanyak 4 citra, yaitu 1 citra terklasifikasi *aphids*, 1 citra terklasifikasi *healthy*, dan 2 terklasifikasi *bacterial blight*.
3. Kelas *healthy* semua data terklasifikasi benar dengan total 60 citra.
4. Terdapat 53 citra *bacterial blight* terdeteksi dengan benar dan memiliki kesalahan prediksi yang berjumlah 7 citra, yaitu 3 citra terklasifikasi *aphids*, 1 citra terklasifikasi *healthy*, dan 3 citra terklasifikasi *target spot*.
5. Terdapat 56 citra dengan benar terklasifikasi *powdery mildew*, namun masih ada kesalahan prediksi berjumlah 4 citra, yaitu 2

citra terklasifikasi *army worm*, 1 citra terklasifikasi *healthy*, dan 1 citra terklasifikasi *bacterial blight*.

6. Terdapat 52 citra *target spot* yang terklasifikasi dengan benar, dan 9 citra yang terklasifikasi tidak tepat.

Dari total 361 citra, jumlah data yang terklasifikasi benar sebanyak 330 citra. Berdasarkan tabel, dapat dilihat bahwa model memiliki performa yang baik dalam melakukan klasifikasi beberapa jenis penyakit dan hama. Namun, terdapat beberapa kesalahan dalam klasifikasi yang terlihat pada bagian di luar diagonal.

Tabel 4.9 *Confusion Matrix* Data Tes

Label	Total Gambar	TP	TN	FN	FP
<i>Aphids</i>	60	53	296	7	5
<i>Army Worm</i>	60	56	294	4	7
Bacterial Blight	60	53	294	7	7
Healthy	60	60	294	0	7
<i>Powdery Mildew</i>	60	56	301	4	0
<i>Target spot</i>	61	52	295	9	5

Untuk memberikan gambaran yang lebih detail mengenai performa model, Tabel 4.9 menyajikan nilai *true positive* (TP), *true negative* (TN), *false negative* (FN) dan *false positive* (FP) untuk masing masing kelas. *Confusion Matrix* ini menunjukkan jumlah prediksi yang benar dan salah untuk setiap label. Berdasarkan hasil tabel ini, dapat diamati bahwa model memiliki jumlah TP yang cukup tinggi berdasarkan total data dari masing-masing kelas. Namun, masih terdapat beberapa nilai FP dan FN yang menunjukkan adanya kesalahan dalam prediksi positif dan negatif.

	precision	recall	f1-score	support
Aphids	0.91	0.88	0.90	60
Army worm	0.89	0.93	0.91	60
Bacterial Blight	0.88	0.88	0.88	60
Healthy	0.90	1.00	0.94	60
Powdery Mildew	1.00	0.93	0.97	60
Target spot	0.91	0.85	0.88	61
accuracy			0.91	361
macro avg	0.92	0.91	0.91	361
weighted avg	0.92	0.91	0.91	361

Gambar 4.17 Performa Klasifikasi *Precision*, *Recall*, dan *F1 Score*

Gambar 4.17 menyajikan performa klasifikasi yang berisi metrik presisi, *recall*, dan *f1 score* dari masing-masing kelas. Metrik ini memberikan gambaran yang lebih komprehensif mengenai performa model dari sisi ketepatan (presisi), sensitivitas (*recall*), dan keseimbangan antara keduanya (*F1 score*). Hasil akurasi yang didapatkan pada data tes ini dengan ukuran 120x120 piksel yaitu 91%. Berikut adalah contoh perhitungan presisi, *recall*, dan *F1 score* untuk label “*Aphids*”:

$$\text{Presisi} = \frac{TP}{TP+FP} = \frac{53}{53+6} = \frac{53}{59} = 0.91$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{53}{53+5} = \frac{53}{58} = 0.88$$

$$\text{F1 score} = \frac{2(\text{Presisi} \times \text{Recall})}{\text{Presisi} + \text{Recall}} = \frac{2(0.88 \times 0.91)}{0.88 + 0.91} = 0.90$$

Berdasarkan perhitungan diatas, kelas “*Aphids*” memiliki rasio presisi sebesar 0.88, *recall* sebesar 0.88, dan *F1 score* sebesar 0.90. Hal ini menunjukkan bahwa performa model dalam melakukan klasifikasi kelas ini memiliki tingkat kesalahan yang rendah, baik dalam FP maupun FN. Untuk memberifikasi kesesuaian antara perhitungan komputasi dan manual, tabel berikut menunjukkan perhitungan manual metrik klasifikasi.

Tabel 4.10 Perhitungan manual *Precision*, *Recall*, dan *F1 score*

Label	Total Gambar	Presisi	<i>Recall</i>	<i>F1 score</i>
Aphids	60	0,91	0,88	0,90
Army Worm	60	0,89	0,93	0,91

Label	Total Gambar	Presisi	<i>Recall</i>	<i>F1 score</i>
Bacterial Blight	60	0,88	0,88	0,88
Healthy	60	0,90	1,00	0,94
Powdery Mildew	60	1,00	0,93	0,97
Target spot	61	0,91	0,85	0,88

Pada Tabel 4.10, nilai presisi, *recall*, dan *F1 score* diatas merupakan hasil dari perhitungan manual dari masing-masing kelas berdasarkan Tabel 4.9. Perbandingan nilai-nilai ini dengan hasil perhitungan komputasi memiliki kesesuaian yang tinggi. Secara keseluruhan, laporan klasifikasi ini menunjukkan bahwa model yang dikembangkan memiliki tingkat keakuratan yang tinggi dalam mengklasifikasikan penyakit dan hama pada tanaman kapas. Hal ini dibuktikan dengan presisi, *recall*, dan *F1 score* yang cukup tinggi pada setiap kelas, model memiliki performa yang baik dalam mengklasifikasi kasus positif secara konsisten. Namun, perlu diperhatikan bahwa masih terdapat beberapa kelas yang memiliki nilai metrik yang lebih rendah dibanding yang lain. Hal ini menunjukkan bahwa model masih memiliki ruang untuk perbaikan.

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil analisis yang telah dilakukan, maka dapat disimpulkan bahwa:

1. Dengan menggunakan metode CNN, 58 citra *Aphids* terklasifikasi dengan benar, sedangkan 7 citra terklasifikasi salah. Terdapat 56 citra *army worm* terklasifikasi dengan benar, namun masih terdapat kesalahan prediksi sebanyak 4 citra. Untuk kelas *healthy* semua data terklasifikasi benar dengan total 60 citra. Terdapat 53 citra *bacterial blight* terdeteksi dengan benar dan memiliki kesalahan prediksi yang berjumlah 7 citra. Terdapat 52 citra *target spot* yang terklasifikasi dengan benar, dan 9 citra yang terklasifikasi tidak tepat. Dari total 361 citra, jumlah data yang terklasifikasi benar sebanyak 330 citra.
2. Proses klasifikasi jenis penyakit dan hama daun tanama kapas menggunakan metode CNN mendapatkan akurasi sebesar 91%. Hal ini mengindikasikan bahwa model telah berhasil mengenali dan mengklasifikasikan penyakit dan hama pada daun kapas dengan tingkat ketepatan yang cukup tinggi.

5.2. Saran

Berikut beberapa saran yang diberikan untuk meningkatkan kualitas pada penelitian ini yang dapat dipertimbangan, antara lain:

1. Penelitian selanjutnya disarankan untuk mengumpulkan lebih banyak data yang berbeda untuk memastikan model mampu menangani variasi yang lebih luas.
2. Memperbanyak jumlah kelas penyakit dan hama yang akan diklasifikasikan untuk meningkatkan kemampuan model dalam mengenali berbagai jenis penyakit dan hama tanaman kapas.
3. Melakukan analisis lebih lanjut mengenai penyebab masih adanya kesalahan dalam klasifikasi. Ini dapat mencakup dalam pengaruh *hyperparameter*, augmentasi, dan kedalaman model CNN.

DAFTAR PUSTAKA

- Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, 8(1), 53. <https://doi.org/10.1186/s40537-021-00444-8>
- Anwar, S., Soomro, S. R., Baloch, S. K., Patoli, A. A., & Kolachi, A. R. (2023). Performance Analysis of Deep Transfer Learning Models for the Automated Detection of Cotton Plant Diseases. *Engineering, Technology and Applied Science Research*, 13(5), 11561–11567. <https://doi.org/10.48084/etasr.6187>
- Bai, Y. (2022). RELU-Function and Derived Function Review. *SHS Web of Conferences*, 144, 02006. <https://doi.org/10.1051/shsconf/202214402006>
- Barbedo, J. G. A. (2016). A review on the main challenges in automatic plant disease identification based on visible range images. *Biosystems Engineering*, 144, 52–60. <https://doi.org/10.1016/j.biosystemseng.2016.01.017>
- Batubara, N. A., & Awangga, R. M. (2020). *Tutorial Object Detection Plate Number With Convolution Neural Network (CNN)* (Vol. 1). Kreatif.
- Campesato, O. (2020). *Artificial intelligence, machine learning, and deep learning*. Mercury Learning and Information.
- Carneiro, T., Medeiros Da Nobrega, R. V., Nepomuceno, T., Bian, G.-B., De Albuquerque, V. H. C., & Filho, P. P. R. (2018). Performance Analysis of Google Colaboratory as a Tool for Accelerating Deep Learning Applications. *IEEE Access*, 6, 61677–61685. <https://doi.org/10.1109/ACCESS.2018.2874767>
- Chen, P., Xiao, Q., Zhang, J., Xie, C., & Wang, B. (2020). Occurrence prediction of cotton pests and diseases by bidirectional long short-term memory networks with climate and atmosphere circulation. *Computers and Electronics in Agriculture*, 176, 105612. <https://doi.org/10.1016/J.COMPAG.2020.105612>
- Cox, K. L., Babilonia, K., Wheeler, T., He, P., & Shan, L. (2019). Return of old foes — recurrence of bacterial blight and Fusarium wilt of cotton. *Current*

- Opinion in Plant Biology*, 50, 95–103.
<https://doi.org/10.1016/j.pbi.2019.03.012>
- Dhamodharan. (2023). *Cotton plant disease* 🌱🍂🍂🌱. Kaggle.
<https://doi.org/https://doi.org/10.34740/KAGGLE/DSV/5127834>
- Dong, Y., Fu, Z., Stankovski, S., Peng, Y., & Li, X. (2021). A Cotton Disease Diagnosis Method Using a Combined Algorithm of Case-Based Reasoning and Fuzzy Logic. *The Computer Journal*, 64(2), 155–168.
<https://doi.org/10.1093/comjnl/bxaa098>
- Gholamalinezhad, H., & Khosravi, H. (2020). *Pooling Methods in Deep Neural Networks, a Review*.
- Hakim, L. (2022). *Bakteri Patogen Tumbuhan*. Syiah Kuala University Press.
- Hardani, Auliya, N. H., Andriani, H., Fardani, R. A., Ustiawaty, J., Utami, E. F., Sukmana, D. J., & Istiqomah, R. R. (2020). *Metode Penelitian Kualitatif & Kuantitatif*. CV. Pustaka Ilmu.
- Husen, S., Sutardjo, H. T., & Zakia, A. (2021). *Teknologi Produksi Tanaman Sayuran*. UMMPress.
- Id, I. D. (2021). *Machine Learning: Teori, Studi Kasus dan Implementasi Menggunakan Python* (Vol. 1). Unri Press.
- Kirana, K. C. (2021). *PENGOLAHAN CITRA DIGITAL: Teori dan Penerapan Pengolahan Citra Digital pada Deteksi Wajah*. Ahlimedia Book.
- Li, R., He, Y., Li, Y., Qin, W., Abbas, A., Ji, R., Li, S., Wu, Y., Sun, X., & Yang, J. (2024). Identification of cotton pest and disease based on CFNet- VoV-GCSP -LSKNet-YOLOv8s: a new era of precision agriculture. *Frontiers in Plant Science*, 15. <https://doi.org/10.3389/fpls.2024.1348402>
- M. Shahbandeh. (2024). *Cotton - statistics & facts* | Statista.
<https://www.statista.com/topics/1542/cotton/#topicOverview>
- Mishra, S., Mishra, D., & Santra, G. H. (2016). Applications of Machine Learning Techniques in Agricultural Crop Production: A Review Paper. *Indian Journal of Science and Technology*, 9(38).
<https://doi.org/10.17485/ijst/2016/v9i38/95032>
- N. Rondon, M., & Lawrence, K. (2021). The fungal pathogen *Corynespora cassiicola*: A review and insights for target spot management on cotton and

- Soya bean. *Journal of Phytopathology*, 169(6), 329–338.
<https://doi.org/10.1111/jph.12992>
- Nurdin, S. Q. (2017). *Mempercepat Panen Sayuran Hidroponik*. Agromedia.
- Putro, E. C., Awangga, R. M., & Andarsyah, R. (2020). *Tutorial Object Detection People With Faster region-Based Convolutional Neural Network (Faster R-CNN)* (Vol. 1). Kreatif.
- Qadir, M., Alam, M., & Farid, S. (2022). A Novel Based Deep Learning Approach for early Diagnosis of Cotton Crop Disease. Dalam *J. inf. commun. technol. robot. appl* (Vol. 13, Nomor 1).
<http://www.jictra.com.pk/index.php/jictra,pISSN:2523-5729,eISSN:2523-5739>
- Raschka, S., Patterson, J., & Nolet, C. (2020). Machine Learning in Python: Main Developments and Technology Trends in Data Science, Machine Learning, and Artificial Intelligence. *Information*, 11(4), 193.
<https://doi.org/10.3390/info11040193>
- Sastrahidayat, I. R. (2017). *Penyakit Tumbuhan yang Disebabkan oleh Jamur*. Universitas Brawijaya Press.
- Thakkar, H., Pingle, A., Kulkarni, S., Saraf, R., & Kulkarni, R. V. (2023). Disease and Pest Detection in crops using Computer Vision: A Comprehensive Study. *Eur. Chem. Bull*, 7022–7028. <https://orcid.org/0000-0001-6614-6297>
- Toscano-Miranda, R., Toro, M., Aguilar, J., Caro, M., Marulanda, A., & Trebilcok, A. (2022). Artificial-intelligence and sensing techniques for the management of insect pests and diseases in cotton: a systematic literature review. *The Journal of Agricultural Science*, 160(1–2), 16–31.
<https://doi.org/10.1017/S002185962200017X>
- Xiao, G., Zhao, P., & Zhang, Y. (2019). A Pivotal Role of Hormones in Regulating Cotton Fiber Development. *Frontiers in Plant Science*, 10.
<https://doi.org/10.3389/fpls.2019.00087>

LAMPIRAN

LAMPIRAN 1

Lampiran 1.1 *python import library*

```
Import Library

import zipfile, os
import shutil
import os.path
from os import path
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.metrics import Recall
from tensorflow.keras.metrics import Precision

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import optimizers

from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam

from sklearn.metrics import classification_report, confusion_matrix
```

Lampiran 1.2 *python persiapan data 1*

```
Persiapan Data 1

#unduh data
!gdown 1cJoKwJNk7VxD_SxKd509lTFZlwXqc6xn

# unzip data
!unzip '/content/dataset_cotton.zip'

# membuat variabel untuk data path
data_path = '/content/Main dataset-20230209T191052Z-001/Main dataset'

# membuat folder train test dan validation
os.mkdir('train')
os.mkdir('test')
os.mkdir('validation')
```

Lampiran 1.3 *python* persiapan data 2

```

● ● ●
Persiapan Data 2

# membagi data menjadi tiga folder secara acak
from sklearn.model_selection import train_test_split
def move_data(data_path, train_ratio=0.75, val_ratio=0.15, test_ratio=0.1):
    """
    Memindahkan dataset menuju train, test, and validation.
    Args:
        data_path: Path menuju direktori dataset utama.
        train_ratio: Rasio data training set
        val_ratio: Rasio data validation set
        test_ratio: Rasio data test set
    Raises:
        ValueError: jika total rasio tidak 1.
    """

    # Memastikan total jumlah rasio 1
    if train_ratio + val_ratio + test_ratio != 1:
        raise ValueError("Sum of train, validation, and test ratios must be equal to 1.")

    # Menginisialisasi variabel untuk menyimpan jalur direktori masing-masing folder dataset
    train_dir = '/content/train'
    val_dir = '/content/validation'
    test_dir = '/content/test'

    os.makedirs(train_dir, exist_ok=True)
    os.makedirs(val_dir, exist_ok=True)
    os.makedirs(test_dir, exist_ok=True)

    for label_dir in os.listdir(data_path):
        # Memperoleh kelas keseluruhan
        label_path = os.path.join(data_path, label_dir)

        # Check if it's a directory and not a file
        if os.path.isdir(label_path):
            # Membagi data berdasarkan label
            data_files = os.listdir(label_path)
            train_files, remaining_files = train_test_split(data_files, test_size=1 -
train_ratio, random_state=None)
            val_files, test_files = train_test_split(remaining_files, test_size=test_ratio /
(val_ratio + test_ratio), random_state=None)

            # Membuat subfolder untuk setiap label pada direktori train, test, dan validation
            os.makedirs(os.path.join(train_dir, label_dir), exist_ok=True)
            os.makedirs(os.path.join(val_dir, label_dir), exist_ok=True)
            os.makedirs(os.path.join(test_dir, label_dir), exist_ok=True)

            # Memindahkan file ke dalam sub folder masing-masing
            for file in train_files:
                shutil.move(os.path.join(label_path, file), os.path.join(train_dir, label_dir,
file))

            for file in val_files:
                shutil.move(os.path.join(label_path, file), os.path.join(val_dir, label_dir,
file))

            for file in test_files:
                shutil.move(os.path.join(label_path, file), os.path.join(test_dir, label_dir,
file))

    # inisialisasi data path
    data_path = "/content/Main dataset-20230209T191052Z-001/Main dataset"
    move_data(data_path)

```

LAMPIRAN 2

Lampiran 2.1 *Python preprocessing dan augmentasi*

```

Data Preprocessing dan Augmentasi

# inisialisasi variabel
size = (120, 120)
batch = 16
train_dir = '/content/train'
test_dir = '/content/test'
validation_dir = '/content/validation'

# Menginisialisasi objek ImageDataGenerator untuk augmentasi dan preprocessing data pada set
# pelatihan
train_datagen = ImageDataGenerator(
    rescale = 1./255,          # Normalisasi piksel gambar dari rentang [0, 255] menjadi [0, 1]
    shear_range=0.2,          # Mengaplikasikan transformasi geser dengan rentang 0.2
    rotation_range=20,         # Memutar gambar dalam rentang 20 derajat untuk variasi
    zoom_range=0.2,           # Memperbesar atau memperkecil gambar dalam rentang 0.2
    horizontal_flip=True,      # Membalik gambar secara horizontal untuk variasi tambahan
)

# Menginisialisasi objek ImageDataGenerator untuk normalisasi data pada set validasi
validation_datagen = ImageDataGenerator(
    rescale=1./255            # Normalisasi piksel gambar dari rentang [0, 255] menjadi [0, 1]
)

# Menginisialisasi objek ImageDataGenerator untuk normalisasi data pada set pengujian
test_datagen = ImageDataGenerator(
    rescale=1./255            # Normalisasi piksel gambar dari rentang [0, 255] menjadi [0, 1]
)
```

Lampiran 2.2 Python ImageDataGenerator data train, validation, dan test

```
ImageDataGenerator

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=size,
    batch_size= batch,
    class_mode='categorical',
    subset='training',
    classes = ['Aphids', 'Army worm', 'Bacterial Blight', 'Healthy', 'Powdery Mildew', 'Target
spot']
)

validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    target_size=size,
    batch_size= batch,
    class_mode='categorical',
    shuffle=False,
    classes=['Aphids', 'Army worm', 'Bacterial Blight', 'Healthy', 'Powdery Mildew', 'Target
spot']
)

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=size,
    batch_size= batch,
    class_mode='categorical',
    classes=['Aphids', 'Army worm', 'Bacterial Blight', 'Healthy', 'Powdery Mildew', 'Target
spot'],
    shuffle = False
)
```


Lampiran 2.3 Python visualisasi data

```

Visualisasi Data

#visualisasi gambar
dataset_path = "/content/train"

# Jumlah sampel per label
num_samples_per_label = 3

# memastikan lokasi dataset
if not os.path.exists(dataset_path):
    print("Error: Dataset path", dataset_path, "does not exist.")
    exit(1)

labels = ["Aphids", "Army worm", "Bacterial Blight", "Healthy", "Powdery Mildew", "Target
spot"]
for label in labels:
    label_path = os.path.join(dataset_path, label)
    if not os.path.isdir(label_path):
        print("Error: Label folder", label_path, "does not exist.")
        exit(1)

    # Pengecekan apakah data ada atau tidak
    image_files = [f for f in os.listdir(label_path) if
os.path.isfile(os.path.join(label_path, f))]
    if len(image_files) < num_samples_per_label:
        print("Warning:", label, "folder has less than", num_samples_per_label, "images. Using
available images.")
        num_samples_per_label = len(image_files) # Adjust based on available images

# menciptakan 6x3 layout
rows, cols = 6, 3

fig, axes = plt.subplots(rows, cols, figsize=(15, 18)) # Adjust figure size for better
visualization

# menampilkan gambar
index = 0
for label in labels:
    label_path = os.path.join(dataset_path, label)
    image_files = [os.path.join(label_path, f) for f in os.listdir(label_path) if
os.path.isfile(os.path.join(label_path, f))]

    for i in range(num_samples_per_label):
        if i >= len(image_files):
            break

        image = plt.imread(image_files[i])
        axes[index // cols, index % cols].imshow(image)
        axes[index // cols, index % cols].set_title(label)
        axes[index // cols, index % cols].axis('off')
        index += 1

plt.tight_layout()
plt.show()

```

LAMPIRAN 3

Lampiran 3.1 *Python* rancangan model

```

Rancangan Model

model = keras.Sequential([
    # bagian feature learning
    keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(120, 120, 3)),
    keras.layers.MaxPooling2D(2, 2),
    keras.layers.Conv2D(64, (3,3), activation='relu'),
    keras.layers.MaxPooling2D(2,2),

    # bagian klasifikasi
    keras.layers.Flatten(),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(256, activation = 'relu'),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(128, activation = 'relu'),
    keras.layers.Dense(6, activation='softmax')
])

# menampilkan ringkasan model
model.summary()

```

Lampiran 3.2 *Python* Pelatihan Model

```

Pelatihan Model

class myCallback(keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs={}):
        if(logs.get('accuracy')> 0.95):
            print("\nAkurasi telah mencapai 95%, hentikan training!")
            self.model.stop_training = True

callbacks = myCallback()

early_stopping = EarlyStopping(monitor='val_loss', patience=3)

model.compile(optimizer= optimizers.Adam(learning_rate = 0.0005),
              loss='categorical_crossentropy',
              metrics=['accuracy', Recall()])

history = model.fit(train_generator,
                    epochs=25,
                    validation_data = validation_generator,
                    callbacks = [callbacks, early_stopping],
                    verbose = 1)

```

Lampiran 3.3 *Python* Visualisasi Proses Pelatihan Model



```
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('accuracy plot')
plt.ylabel('value')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('loss plot')
plt.ylabel('value')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
```

LAMPIRAN 4

Lampiran 4.1 *Python* Uji Model Menggunakan Data Tes

```

Tes Hasil Model

# cetak kelas dan indeksinya
print("Class indices in test generator:", test_generator.class_indices)
print("Classes in test set:", np.unique(test_generator.classes))

# cetak jumlah data dalam tes setiap gambar
for class_name in ['Aphids', 'Army worm', 'Bacterial Blight', 'Healthy', 'Powdery Mildew',
'Target spot']:
    class_path = os.path.join(test_dir, class_name)
    print(f"{class_name}: {len(os.listdir(class_path))} images")

# prediksi data baru
predictions = model.predict(test_generator)

```

Lampiran 4.2 *Python* Visualisasi Laporan Klasifikasi

```

Visualisasi Hasil Prediksi

# menampilkan hasil klasifikasi data tes
y_true = test_generator.classes
print(classification_report(y_true, predictions.argmax(axis=1)))
print(confusion_matrix(y_true, predictions.argmax(axis=1)))

import seaborn as sns

# Inisialisasi variabel prediksi dan nilai sebenarnya
y_true = test_generator.classes
y_pred = predictions.argmax(axis=1)

# Kalkulasi dengan confusion matrix
cm = confusion_matrix(y_true, y_pred)

# Definisi label
labels = ["Aphids", "Army worm", "Bacterial Blight", "Healthy", "Powdery Mildew", "Target
spot"]

# visualisasi heatmap
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=labels, yticklabels=labels)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix Heatmap')
plt.show()

print(classification_report(y_true, y_pred, target_names=labels))

```

Lampiran 4.3 Python Uji Manual

```
import numpy as np
from google.colab import files
from keras.preprocessing import image
import matplotlib.pyplot as plt
%matplotlib inline

uploaded = files.upload()

for fn in uploaded.keys():
    path = fn
    img = image.load_img(path, target_size=size)
    imgplot = plt.imshow(img)
    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)
    x /= 255.0

    images = np.vstack([x])
    classes = model.predict(images, batch_size=10)[0]

    print(fn)
    print("Raw Predictions:", classes)

    sorted_indices = np.argsort(classes)[::-1]

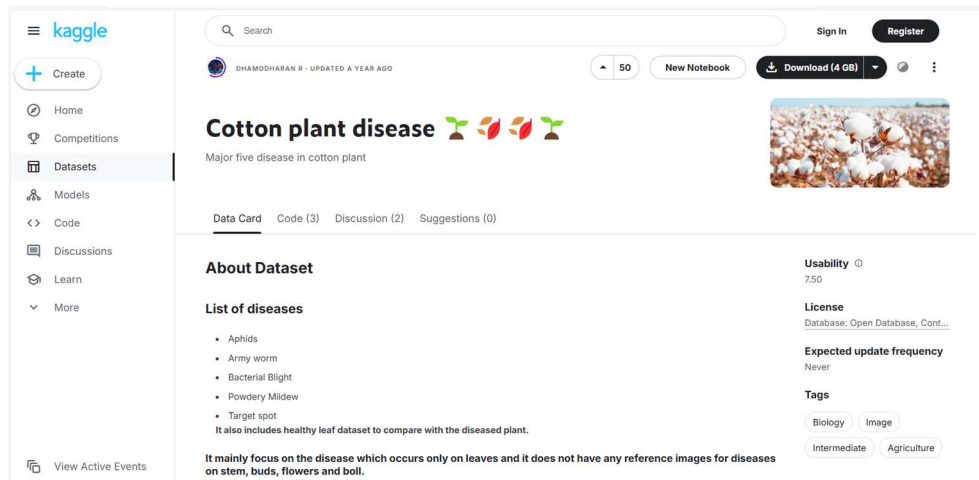
    class_names = ['Aphids', 'Army Worm', 'Bacterial Blight', 'Healthy', 'Powdery Mildew',
'Target Spot']

    top_classes = [(class_names[i], classes[i]*100) for i in sorted_indices[:6]]

    print("Top Predicted Classes:")
    for class_name, probability in top_classes:
        print(f"{class_name}: {probability:.2f}%")
```

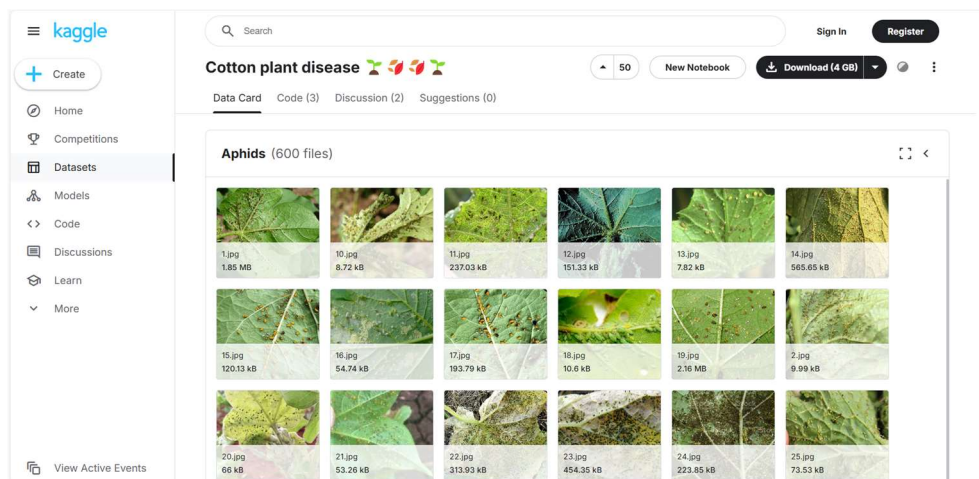
LAMPIRAN 5

Lampiran 5.1 Website Sumber Data



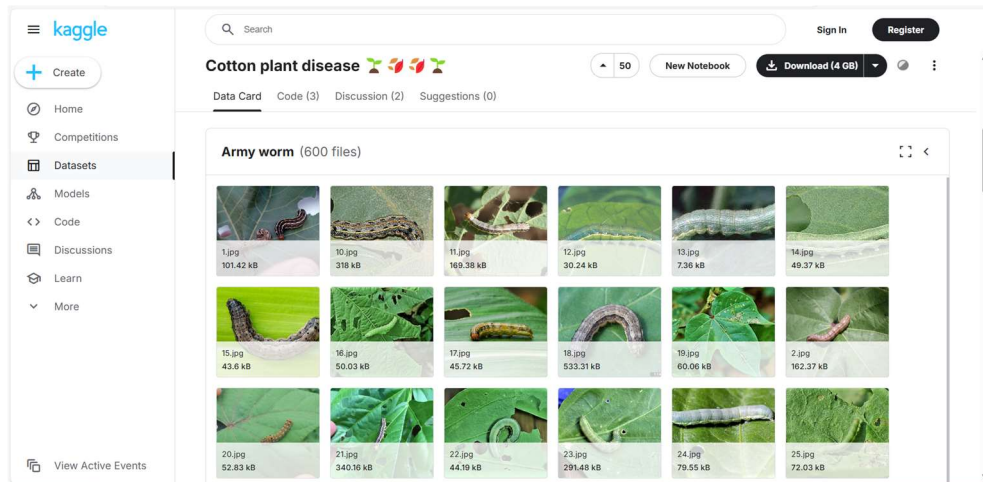
The screenshot shows the Kaggle dataset page for 'Cotton plant disease' by DHAMODHARAN R. The page includes a search bar, navigation links (Home, Competitions, Datasets, Models, Code, Discussions, Learn, More), and a sidebar with 'View Active Events'. The dataset title is 'Cotton plant disease' with a subtitle 'Major five disease in cotton plant'. It has 3 code files, 2 discussions, and 0 suggestions. The dataset size is 4 GB. The 'About Dataset' section lists diseases: Aphids, Army worm, Bacterial Blight, Powdery Mildew, and Target spot. It also mentions a healthy leaf dataset for comparison. The 'Usability' is 7.50, the license is 'Open Database, Cont...', and the expected update frequency is 'Never'. Tags include Biology, Image, Intermediate, and Agriculture.

Lampiran 5.2 Label *Aphids*

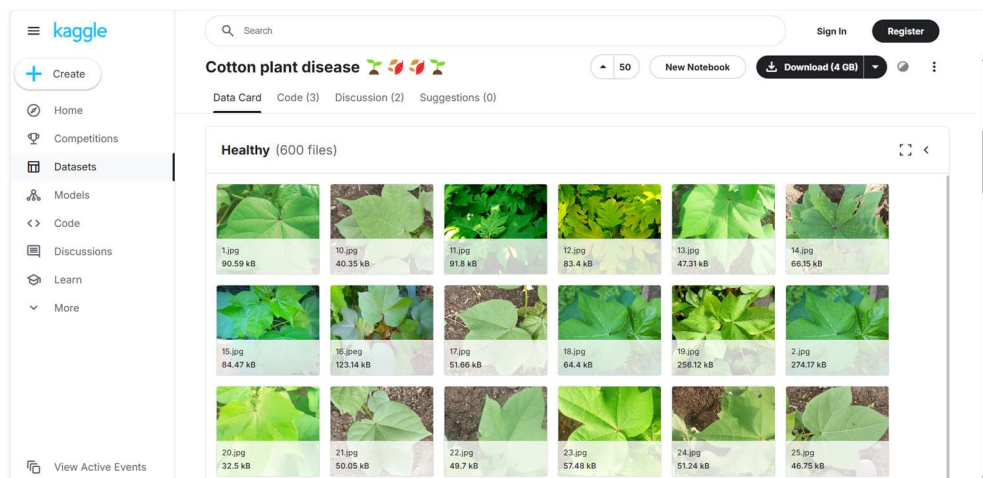


The screenshot shows the 'Aphids' label for the 'Cotton plant disease' dataset. It displays 600 files in a grid format. Each file is a small image of a cotton leaf with an aphid, labeled with a number (1.jpg to 25.jpg) and its size. The files are arranged in three rows of six, with the last row containing only five files.

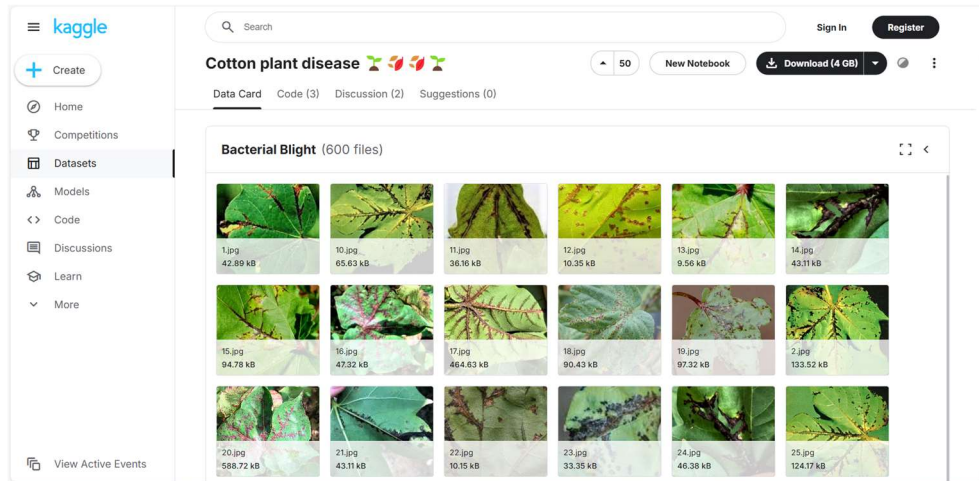
Lampiran 5.3 Label *Army Worm*



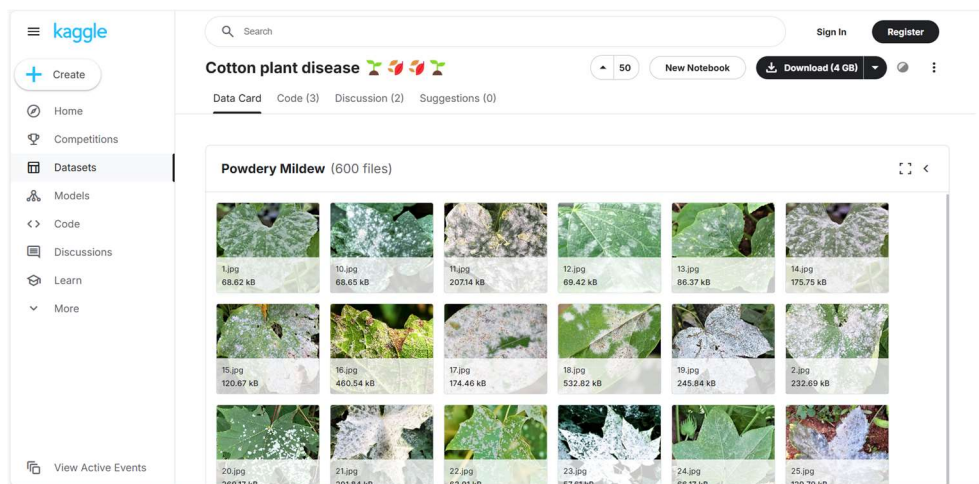
Lampiran 5.4 Label *Healthy*



Lampiran 5.5 Label *Bacterial Blight*



Lampiran 5.6 Label *Powdery Mildew*



Lampiran 5.7 Label *Target Spot*

